Generación de Escenarios Movilidad

Empresa EFSS

ESPECIFICACIÓN DE REQUERIMIENTOS DE SOFTWARE

2026-10-13

versión 2.0



Erick Salazar Suarez
Felipe Garrido Flores
Samuel Campos Capañas
Santiago Chitiva Contreras

Historial de cambios

Versión	Fecha	Sección modificada	Descripción de cambios	Responsables
1.0	2026-05-31	Todas	Versión inicial del SRS: pipeline ETL (F1–F5) y módulo web (F6–F7).	Equipo EFSS
2.0	2026-10-13	1.2, 1.3, 2.1, 2.2, 2.5, 2.8, 3.1, 3.2, 3.3, 3.4, 3.5, 3.6, 4.1, 4.4, 5.3, 5.4	Se actualiza el documento al estado de la rama `feat/fases-b-f-estilos-cierres`. Cada requerimiento indica su estado (Implementado, Parcial, Pendiente) y su módulo real. Se agregan: • demanda parametrizable (RF-021 a RF-025) y progreso, cancelación y animación de viajes (RF-026, RF-027); • intersecciones y semaforización (F8, RF-028 a RF-032); • simulador 3D comparativo de semáforo propuesto (F9, RF-033 a RF-042) y transporte público con GTFS (RF-043, pendiente); • RD-003, RD-004 y RNF-USA-001, con sus casos de prueba CP-029 a CP-042. Se reescriben las interfaces de usuario, software y comunicaciones (visor web, WebSocket, TraCI).	Equipo EFSS

Tabla 1: Historial de cambios.

Contenido

HISTORIAL DE CAMBIOS **1**
CONTENIDO **2**
LISTA DE TABLAS **4**
LISTA DE ILUSTRACIONES **6**
1. INTRODUCCION **7**
1.1. Propósito 7
1.2. Alcance 7
1.3. Definiciones, Acrónimos y Abreviaciones 8
1.4. Referencias 10
1.5. Apreciación Global 11
2. DESCRIPCIÓN GLOBAL **11**
2.1. Perspectiva del Producto 11
2.1.1. Interfaces con el Sistema 12
2.1.2. Interfaces con el Usuario 14
2.1.3. Interfaces con el Hardware 14
2.1.4. Interfaces con el Software 15
2.1.5. Interfaces de Comunicación 16
2.1.6. Restricciones de Memoria 16
2.1.7. Operaciones 16
2.1.8. Requerimientos de Adaptación del Sitio 16
2.2. Funciones del Producto 17
2.3. Características del Usuario 18
2.4. Restricciones 19
2.4.1. Restricciones Generales 19
2.4.2. Restricciones de Software 19
2.4.3. Restricciones de Hardware 19
2.5. Modelo del Dominio 19
2.6. Suposiciones y Dependencias 22
2.7. Distribución de Requerimientos 22
3. REQUERIMIENTOS ESPECÍFICOS **23**
3.1. Requerimientos de Interfaces Externas 24
3.1.1. Interfaces con el Usuario 24
3.1.2. Interfaces con el Hardware 25
3.1.3. Interfaces con el Software 25
3.1.4. Interfaces de Comunicaciones 26
3.2. Características del Producto de Software (Requerimientos Funcionales) 27
3.2.1. F1 – Ingestión y Validación de Datos de Entrada 27
3.2.2. F2 – Construcción de Matrices Origen-Destino 30
3.2.3. F3 – Generación de Red Vial y Zonas TAZ 31
3.2.4. F4 – Generación de Archivos de Simulación 32
3.2.5. F5 – Variación Paramétrica y Escenarios Hipotéticos 33
3.2.6. F6 – Módulo Web: Visualización y Parametrización 36
3.2.7. F7 – Validación y Reportes 40
3.2.8. F8 – Intersecciones y Semaforización 41
3.2.9. F9 – Simulador 3D Comparativo de Semáforo Propuesto 44

3.3. Requerimientos de Desempeño 50
3.4. Restricciones de Diseño 52
3.5. Atributos del Sistema de Software (Requerimientos No Funcionales) 52
3.5.1. Confiabilidad 52
3.5.2. Disponibilidad 53
3.5.3. Seguridad 54
3.5.4. Mantenibilidad 54
3.5.5. Portabilidad 55
3.5.6. Reproducibilidad 56
3.5.7. Observabilidad 57
3.5.8. Usabilidad 57
3.6. Requerimientos de la Base de Datos 58

4. PROCESO DE INGENIERÍA DE REQUERIMIENTOS 59
4.1. Elicitación de Requerimientos 59
4.1.1. Entrevistas con el Director del Proyecto 59
4.1.2. Análisis de Documentación 60
4.1.3. Exploración de Datos (CRISP-DM – Fase 1) 60
4.2. Análisis y Priorización de Requerimientos 60
4.2.1. Resolución de Conflictos 60
4.2.2. Esquema de Priorización 60
4.3. Especificación y Documentación 61
4.4. Trazabilidad de Requerimientos 61
4.4.1. Trazabilidad hacia el origen 61
4.4.2. Trazabilidad hacia casos de prueba 62
4.5. Control de Cambios a los Requerimientos 62
4.5.1. Flujo de Control de Cambios 63
4.5.2. Versionamiento de Requerimientos 63
4.5.3. Restricciones al Control de Cambios 63

5. PROCESO DE VERIFICACIÓN 64
5.1. Verificación del Documento SRS 64
5.1.1. Técnica de Revisión por Pares 64
5.1.2. Revisión con el Director del Proyecto 64
5.2. Verificación de Requerimientos Individuales 65
5.3. Plan de Casos de Prueba 65
5.3.1. Pruebas de Requerimientos Funcionales 65
5.3.2. Pruebas de Requerimientos No Funcionales 70
5.4. Criterios de Aceptación del Sistema 71
5.5. Proceso de Registro de Defectos 72

6. ANEXOS 72
6.1. Descripción de la Encuesta de Movilidad de Bogotá 2023 72
6.2. Posición de Bogotá en el TomTom Traffic Index 2025 73

Lista de tablas

Tabla 1 Historial de cambios. 1
Tabla 2 Definiciones, acrónimos y abreviaciones del proyecto. 8
Tabla 3 Interfaces del sistema con sistemas externos. 12
Tabla 4 Requisitos mínimos de hardware. 14
Tabla 5 Interfaces con el software. 15
Tabla 6 Funciones del producto y artefactos generados. 18
Tabla 7 Características de las clases de usuario. 18
Tabla 8 Entidades del modelo del dominio. 20
Tabla 9 Distribución de requerimientos por módulo (v2.0). 22
Tabla 10 Comandos de línea de comandos del sistema. 25
Tabla 11 Requisitos mínimos de hardware. 25
Tabla 12 Interfaces de software del sistema. 25
Tabla 13 Interfaces de comunicaciones. 26
Tabla 14 Requerimiento RF-001. 27
Tabla 15 Requerimiento RF-002. 28
Tabla 16 Requerimiento RF-003. 28
Tabla 17 Requerimiento RF-004. 29
Tabla 18 Requerimiento RF-005. 29
Tabla 19 Requerimiento RF-006. 30
Tabla 20 Requerimiento RF-007. 30
Tabla 21 Requerimiento RF-008. 31
Tabla 22 Requerimiento RF-009. 31
Tabla 23 Requerimiento RF-010. 32
Tabla 24 Requerimiento RF-011. 32
Tabla 25 Requerimiento RF-012. 33
Tabla 26 Requerimiento RF-013. 33
Tabla 27 Requerimiento RF-021. 34
Tabla 28 Requerimiento RF-022. 35
Tabla 29 Requerimiento RF-023. 35
Tabla 30 Requerimiento RF-024. 35
Tabla 31 Requerimiento RF-025. 36
Tabla 32 Requerimiento RF-017. 36
Tabla 33 Requerimiento RF-018. 37
Tabla 34 Requerimiento RF-019. 38
Tabla 35 Requerimiento RF-020. 38
Tabla 36 Requerimiento RF-026. 39
Tabla 37 Requerimiento RF-027. 39
Tabla 38 Requerimiento RF-014. 40
Tabla 39 Requerimiento RF-015. 40
Tabla 40 Requerimiento RF-016. 41
Tabla 41 Requerimiento RF-028. 41
Tabla 42 Requerimiento RF-029. 42
Tabla 43 Requerimiento RF-030. 42
Tabla 44 Requerimiento RF-031. 43
Tabla 45 Requerimiento RF-032. 43
Tabla 46 Requerimiento RF-033. 44

Tabla 47 Requerimiento RF-034. 45
Tabla 48 Requerimiento RF-035. 45
Tabla 49 Requerimiento RF-036. 46
Tabla 50 Requerimiento RF-037. 46
Tabla 51 Requerimiento RF-038. 47
Tabla 52 Requerimiento RF-039. 47
Tabla 53 Requerimiento RF-040. 48
Tabla 54 Requerimiento RF-041. 48
Tabla 55 Requerimiento RF-042. 49
Tabla 56 Requerimiento RF-043. 49
Tabla 57 Requerimiento RD-001. 50
Tabla 58 Requerimiento RD-002. 50
Tabla 59 Requerimiento RD-003. 51
Tabla 60 Requerimiento RD-004. 51
Tabla 61 Requerimiento RNF-CON-001. 52
Tabla 62 Requerimiento RNF-CON-002. 53
Tabla 63 Requerimiento RNF-DISP-001. 53
Tabla 64 Requerimiento RNF-SEG-001. 54
Tabla 65 Requerimiento RNF-MAN-001. 54
Tabla 66 Requerimiento RNF-PORT-001. 55
Tabla 67 Requerimiento RNF-PORT-002. 55
Tabla 68 Requerimiento RNF-REP-001. 56
Tabla 69 Requerimiento RNF-OBS-001. 57
Tabla 70 Requerimiento RNF-USA-001. 57
Tabla 71 Tipos de datos almacenados por el pipeline. 58
Tabla 72 Esquema de priorización MoSCoW. 61
Tabla 73 Convención de identificadores de requerimientos. 61
Tabla 74 Trazabilidad de requerimientos seleccionados hacia su origen. 62
Tabla 75 Flujo de control de cambios a los requerimientos. 63
Tabla 76 Criterios de verificación del documento SRS. 64
Tabla 77 Métodos de verificación de requerimientos. 65
Tabla 78 Plan de casos de prueba para requerimientos funcionales. 65
Tabla 79 Plan de casos de prueba para requerimientos no funcionales. 70
Tabla 80 Proceso de registro y gestión de defectos. 72

Lista de ilustraciones

Figura 1 Gráfico modelo de dominio. 20

1. Introduccion

1.1. Propósito

El presente documento constituye la Especificación de Requisitos de Software (SRS) del proyecto **Generación de Escenarios Movilidad**, desarrollado para la materia proyecto de grado.

El propósito de este SRS es definir de manera precisa y verificable los requerimientos funcionales y no funcionales del sistema, de forma que sirva como contrato técnico entre el equipo de desarrollo y el director del proyecto. El documento describe qué debe hacer el sistema, bajo qué restricciones debe operar y cómo se verificará su correcto funcionamiento.

La audiencia de este documento incluye:

- **Equipo de desarrollo:** como referencia central para la implementación, las pruebas y la toma de decisiones técnicas a lo largo del ciclo de vida del proyecto.
- **Director del proyecto (Ing. Andrés Calderón Romero):** como revisor y aprobador de los requerimientos antes de la fase de construcción.
- **Jurados evaluadores:** como insumo para la evaluación académica del proyecto de grado.
- **Investigadores y analistas de movilidad:** potenciales usuarios del sistema en futuras iteraciones del producto.

El alcance del documento cubre la versión 2.0 del sistema, que corresponde al estado de la rama de desarrollo actual más los requerimientos aún por desarrollar (marcados como **Pendiente** en la Sección 3). El caso de validación está delimitado a la localidad de Usaquén en Bogotá, Colombia.

1.2. Alcance

El producto de software especificado en este documento es un módulo de procesamiento de datos que automatiza la transformación de fuentes de datos de movilidad urbana disponibles para Bogotá en archivos de entrada válidos para el simulador de tráfico SUMO (Simulation of Urban MObility).

Nombre del producto: Generación de Escenarios de Movilidad Urbana – Usaquén (servicio de simulación `simulation-service` y visor web `kepler`).

Funcionalidades principales que incluirá el sistema:

- Ingesta y limpieza de la Encuesta de Movilidad de Bogotá 2023 y de los aforos vehiculares en Hora de Máxima Demanda (HMD).
- Descarga y procesamiento de la red vial de Usaquén desde OpenStreetMap mediante osmium y netconvert, generando el archivo `.net.xml` requerido por SUMO.
- Construcción de zonas de análisis de tráfico (TAZ) a partir de las Unidades de Planeamiento Zonal (UPZ) de Usaquén, generando el archivo `.taz.xml`.
- Producción de archivos de rutas (`.rou.xml`) y tipos de vehículo (`.vtype.xml`) compatibles con duarouter y SUMO.
- Variación paramétrica de las configuraciones de entrada para la generación de escenarios hipotéticos de movilidad.
- Validación estadística de los resultados mediante el estadístico GEH y RMSE, contrastando los volúmenes simulados con los aforos reales.
- Módulo web que permite visualizar de forma interactiva los resultados producidos por SUMO (métricas de congestión, tiempos de viaje, flujos vehiculares por segmento) y parametrizar nuevas simulaciones desde una interfaz gráfica, sin necesidad de editar archivos XML ni interactuar con la línea de comandos.

- Demanda parametrizable con presets de transferencia modal (encuesta, más vehículos, más peatones, día sin carro), conteos por modo, semilla, estilos de conducción y cierres viales; simulaciones precalculadas de cada preset; avance real y cancelación de corridas.
- Módulo de intersecciones: catálogo de semáforos con volúmenes por acceso, edición de programas, cálculo de Webster, recorte de zonas, comparación de programas y priorización de cruces sin semáforo.
- Simulador 3D comparativo: vista en pantalla partida de un cruce sin semáforo y del mismo cruce con un semáforo propuesto, con la red 3D real, edificios, seguimiento en tercera persona de cualquier vehículo o peatón, HUD de velocidad, riesgo (TTC, DRAC, PET) y decisiones, y minimapa 2D.

Utilidad y objetivos del producto:

El módulo busca eliminar la barrera técnica que impide a planificadores y analistas de movilidad urbana utilizar SUMO con datos reales de Bogotá. Al automatizar el pipeline de extremo a extremo, se reduce de horas a minutos el tiempo necesario para preparar una simulación, permitiendo explorar escenarios hipotéticos como cambios en infraestructura vial o en estrategias de control de tráfico sin intervención manual en la preparación de datos.

Relación con la estrategia organizacional:

El producto está alineado con las necesidades de organizaciones como la Secretaría Distrital de Movilidad de Bogotá, o usuarios como ingenieros de movilidad e infraestructura, los cuales requieren de herramientas sencillas y fiables en cuanto a toma de decisiones frente a implementación de estrategias o políticas de movilidad en Bogotá. Según el TomTom Traffic Index 2025, Bogotá ocupa el séptimo lugar global en congestión vehicular, con velocidades promedio en horas pico que apenas superan los 15 km/h, lo que evidencia la urgencia de contar con herramientas de análisis y simulación accesibles.

Funcionalidades excluidas de esta versión:

- Integración en tiempo real con APIs de tráfico comerciales.
- Despliegue en sistemas de producción institucionales.
- Calibración del modelo para localidades distintas a Usaquén.
- Programas semafóricos reales de la Secretaría Distrital de Movilidad: los programas provienen de netconvert/OpenStreetMap.
- Datos de siniestralidad en la priorización de cruces (criterio reservado).

1.3. Definiciones, Acrónimos y Abreviaciones

A continuación se presentan los términos clave utilizados a lo largo del documento. Este glosario complementa y extiende el definido en el documento SPMP del proyecto.

Término / Acrónimo	Definición
SUMO	Simulation of Urban MObility. Simulador de tráfico microscópico de código abierto desarrollado por el DLR (German Aerospace Center). Permite modelar sistemas de tráfico intermodal incluyendo vehículos, transporte público y peatones.
SRS	Software Requirements Specification. Especificación de Requisitos del Software. Documento que describe de forma completa el comportamiento del sistema a desarrollar.

Término / Acrónimo	Definición
SPMP	Software Project Management Plan. Plan de Administración del Proyecto de Software. Documento que define la planificación, organización y control del proyecto.
ETL	Extract, Transform, Load. Pipeline de procesamiento de datos estructurado en tres etapas: extracción desde fuentes originales, transformación al formato requerido y carga en el sistema destino.
O/D	Matriz Origen-Destino. Estructura de datos que cuantifica el número de viajes realizados entre pares de zonas geográficas en un período de tiempo definido.
TAZ	Traffic Analysis Zone. Unidad espacial básica del modelo de demanda de transporte. En este proyecto corresponde a las UPZ de la localidad de Usaquén.
UPZ	Unidad de Planeamiento Zonal. División administrativa de Bogotá utilizada como unidad de análisis geográfico (TAZ) en este proyecto.
HMD	Hora de Máxima Demanda. Período del día con mayor volumen vehicular, típicamente en jornada AM (mañana) o PM (tarde).
GEH	Geoffrey E. Havers. Estadístico de validación de modelos de tráfico. Un valor GEH < 5 indica una buena calibración del modelo.
RMSE	Root Mean Square Error. Raíz del Error Cuadrático Medio. Métrica de precisión numérica utilizada para cuantificar la diferencia entre valores simulados y observados.
OSM	OpenStreetMap. Proyecto colaborativo de cartografía en línea que provee datos geoespaciales de código abierto. Fuente de la red vial base del modelo.
TraCI	Traffic Control Interface. API oficial de SUMO para controlar y consultar el estado de una simulación en tiempo real mediante Python u otros lenguajes.
netconvert	Herramienta del ecosistema SUMO que convierte redes viales desde formatos externos (OSM, VISUM, etc.) al formato nativo `.net.xml` de SUMO.
duarouter	Herramienta del ecosistema SUMO que asigna rutas óptimas a los viajes definidos en una matriz O/D, generando el archivo `.rou.xml`.
CRISP-DM	Cross-Industry Standard Process for Data Mining. Metodología iterativa de ciencia de datos adoptada para las fases 1 y 2 del proyecto.
Scrum	Marco ágil de desarrollo de software basado en sprints iterativos cortos. Adoptado a partir de la Fase 3 del proyecto.
PEP 8	Guía de estilo para código Python de la Python Software Foundation. Estándar de codificación obligatorio en el proyecto.
API	Application Programming Interface. Interfaz de programación que permite la comunicación entre componentes de software.
DLR	German Aerospace Center (Deutsches Zentrum für Luft- und Raumfahrt). Organización responsable del desarrollo principal de SUMO.

Término / Acrónimo	Definición
WBS	Work Breakdown Structure. Estructura jerárquica de descomposición de las tareas del proyecto.
CI	Continuous Integration. Integración Continua. Práctica de automatizar la ejecución de pruebas en cada commit al repositorio, implementada mediante GitHub Actions.
REST API	Representational State Transfer Application Programming Interface. Estilo de arquitectura para servicios web utilizado por el módulo web para exponer los resultados de SUMO al frontend.
Frontend	Capa de presentación del módulo web. Interfaz gráfica accesible desde el navegador que permite visualizar resultados de simulación y configurar nuevos escenarios sin interacción directa con la línea de comandos.
WebSocket	Protocolo de comunicación bidireccional sobre una conexión TCP. Lo usa el simulador 3D para recibir frames y enviar comandos en vivo.
tlLogic	Elemento de la red de SUMO que define el programa de un semáforo: sus fases, con duración y estado por conexión controlada.
Webster	Método clásico para calcular el ciclo óptimo de un semáforo, $C = \frac{1{,}5L+5}{1-Y}$, a partir de las pérdidas por fase y de la razón entre flujo y flujo de saturación de los accesos críticos.
SSM	Surrogate Safety Measures. Indicadores sustitutos de seguridad que estiman la cercanía a un choque sin que ocurra (TTC, DRAC, PET).
TTC	Time To Collision. Tiempo hasta alcanzar al vehículo de adelante si ambos mantienen su velocidad. Crítico por debajo de 1,5 s.
DRAC	Deceleration Rate to Avoid Collision. Frenado necesario para no alcanzar al vehículo de adelante. Crítico por encima de 3,35 m/s^2.
PET	Post-Encroachment Time. Diferencia de tiempo entre el paso de dos actores por el mismo punto de conflicto. Crítico por debajo de 1,5 s.
Lockstep	Ejecución de dos simulaciones avanzando exactamente el mismo intervalo en cada paso, para compararlas instante a instante.
Ego	Actor (vehículo o peatón) que el usuario sigue con la cámara en tercera persona en el simulador 3D.
FCD	Floating Car Data. Salida de SUMO con la posición de cada actor en cada paso; base de la animación de viajes en el visor.
kepler.gl / three.js	Bibliotecas de visualización del visor: kepler.gl para el mapa animado de viajes y three.js para el render 3D.

Tabla 2: Definiciones, acrónimos y abreviaciones del proyecto.

1.4. Referencias

[1] IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE, 1998.

[2] IEEE Std 1058-1998, *IEEE Standard for Software Project Management Plans*, IEEE, 1998.

[3] P. A. Lopez et al., «Microscopic Traffic Simulation using SUMO,» in *Proc. 21st IEEE International Conference on Intelligent Transportation Systems (ITSC)*, Maui, HI, USA, 2018, pp. 2575–2582. DOI: 10.1109/ITSC.2018.8569938.

[4] Secretaría Distrital de Movilidad de Bogotá, *Encuesta de Movilidad de Bogotá 2023*, Alcaldía Mayor de Bogotá, 2023. [En línea]. Disponible en: https://www.movilidadbogota.gov.co
[5] TomTom International BV, *TomTom Traffic Index 2025*, 2025. [En línea]. Disponible en: https://www.tomtom.com/traffic-index/
[6] OpenStreetMap contributors, *OpenStreetMap*, 2024. [En línea]. Disponible en: https://www.openstreetmap.org
[7] Eclipse Foundation, *Eclipse SUMO — Simulation of Urban MObility: User Documentation*, versión 1.19, 2023. [En línea]. Disponible en: https://sumo.dlr.de/docs/
[8] G. Van Rossum y el equipo de Python, *PEP 8 — Style Guide for Python Code*, Python Software Foundation, 2001. [En línea]. Disponible en: https://peps.python.org/pep-0008/

1.5. Apreciación Global

El presente documento está organizado en seis secciones principales que cubren de forma progresiva todos los aspectos del sistema especificado:

La **Sección 1 – Introducción** (actual) presenta el propósito, el alcance y los términos fundamentales del documento, proporcionando al lector el marco conceptual necesario para interpretar correctamente el resto del contenido.

La **Sección 2 – Descripción Global** describe el sistema desde una perspectiva de alto nivel: su contexto de operación, las interfaces con sistemas externos (SUMO, OSM, encuestas de movilidad), los tipos de usuarios que interactuarán con él y las restricciones generales de diseño e implementación.

La **Sección 3 – Requerimientos Específicos** constituye el núcleo técnico del SRS. En ella se detallan los requerimientos funcionales organizados por capacidades del pipeline ETL, los requerimientos no funcionales (rendimiento, confiabilidad, mantenibilidad, portabilidad) y las interfaces del sistema.

La **Sección 4 – Proceso de Ingeniería de Requerimientos** documenta la metodología utilizada para elicitar, analizar, especificar y validar los requerimientos, describiendo las técnicas empleadas y los artefactos producidos durante este proceso.

La **Sección 5 – Proceso de Verificación** describe el plan de validación del sistema, especificando los criterios de aceptación, las métricas de calidad (GEH, RMSE) y los casos de prueba asociados a los requerimientos.

La **Sección 6 – Anexos** incluye material complementario como diagramas de flujo del pipeline, ejemplos de los formatos de entrada y salida, y trazabilidad entre requerimientos y casos de prueba.

2. Descripción Global

En esta sección se describe el contexto general del producto de software, los factores que lo rodean y las características de los usuarios que lo utilizarán. Esta descripción no constituye una especificación formal de requerimientos, sino el marco de referencia sobre el cual se construyen los requerimientos específicos de la Seccion 3.

2.1. Perspectiva del Producto

El **Pipeline** es un producto de software nuevo, sin precedente directo en el contexto de Bogotá. Si bien existen herramientas internacionales para la conversión de datos de movilidad a formatos de simulación (como el convertidor VISUM-SUMO del propio DLR), ninguna de ellas está adaptada para las fuentes de datos locales: la Encuesta de Movilidad de Bogotá, los aforos vehiculares HMD de la Secretaría Distrital de Movilidad y la red vial de OpenStreetMap para la ciudad.

El sistema se justifica por la brecha existente entre la disponibilidad de datos y la capacidad de usarlos. Bogotá ocupa el séptimo lugar mundial en congestión vehicular según el TomTom Traffic Index 2025, con velocidades promedio en horas pico que apenas superan los 15 km/h. Pese a ello, herramientas de simulación de clase mundial como SUMO no son usadas sistemáticamente por los analistas de movilidad de la ciudad, precisamente porque la transformación de los datos reales al formato que SUMO requiere es un proceso manual, técnicamente complejo y que puede tomar horas de trabajo especializado.

El sistema desarrollado está compuesto por dos módulos complementarios. El primero es un pipeline de procesamiento de datos que automatiza la transformación de extremo a extremo: recibe como insumos la encuesta de movilidad y los datos de infraestructura vial, y produce los archivos listos para ejecutar una simulación en SUMO. El segundo es un módulo web que permite visualizar los resultados producidos por SUMO de forma interactiva y configurar nuevos escenarios de simulación desde una interfaz gráfica, eliminando la necesidad de editar archivos de configuración XML manualmente. El beneficio principal para la Secretaría Distrital de Movilidad y para investigadores es poder explorar escenarios hipotéticos de movilidad sin intervención manual en la preparación de datos ni en la interpretación de resultados.

2.1.1. Interfaces con el Sistema

El pipeline interactúa con los siguientes sistemas externos. Ninguno de ellos forma parte del sistema a desarrollar; todos son proveedores de datos o herramientas que el pipeline invoca como procesos externos.

Sistema externo	Tipo de interfaz	Descripción de la interacción
OpenStreetMap	Descarga de datos (HTTPS)	El pipeline descarga el dump OSM de Bogotá o consulta la API Overpass para extraer la red vial del polígono de Usaquén. Esta es la única comunicación de red del sistema; ocurre una única vez durante la fase de ingestión (F1).
osmium-tool	Proceso externo (CLI)	Invocado por el pipeline para recortar el dump OSM al polígono de Usaquén y exportar el resultado en formato `.osm.pbf`. Se llama mediante subproceso de Python.
netconvert (SUMO)	Proceso externo (CLI)	Herramienta oficial de SUMO invocada para convertir la red OSM al formato `.net.xml`. El pipeline construye el comando con los parámetros definidos en el archivo de configuración y captura su salida estándar para el log.
duarouter (SUMO)	Proceso externo (CLI)	Herramienta oficial de SUMO invocada para asignar rutas a los viajes definidos en la matriz O/D, produciendo el archivo `.rou.xml`.

Sistema externo	Tipo de interfaz	Descripción de la interacción
		Su ejecución es iniciada y monitoreada por el pipeline.
SUMO	Proceso externo (CLI)	Motor de simulación que consume los artefactos producidos por el pipeline (`.net.xml`, `.taz.xml`, `.rou.xml`, `.vtype.xml`) para ejecutar la simulación. El pipeline no controla SUMO en tiempo real; la ejecución de SUMO y la recolección de resultados de aforo es un paso posterior al pipeline.
Encuesta de Movilidad de Bogotá 2023	Archivo XLSX (lectura)	Fuente primaria de datos de demanda. El pipeline lee el archivo XLSX directamente desde disco. No hay comunicación en red con la Secretaría Distrital de Movilidad; el archivo se obtiene previamente del portal de datos abiertos de Bogotá.
Base de datos de aforos HMD	Archivo tabular (lectura)	Fuente de datos de volúmenes vehiculares reales en Hora de Máxima Demanda para los puntos de aforo de Usaquén. Usada exclusivamente en la fase de validación (F5) para calcular el estadístico GEH.
TraCI (SUMO)	Socket TCP local	La API controla SUMO paso a paso para medir el avance de las corridas y, en el simulador 3D, para correr dos simulaciones en lockstep, leer el estado de cada actor e inyectar el actor seguido.
`cutRoutes.py` / `randomTrips.py`	Proceso externo (Python)	Herramientas de SUMO para recortar la demanda real a una zona y generar el tráfico de fondo por modo de las comparaciones 3D.
Microsoft Building Footprints	Archivo GeoJSON (lectura)	Huellas de edificios de Usaquén (`data/processed/edificios_ms.geojson`) que se extruyen en la escena 3D.
Mapbox / OpenStreetMap tiles	HTTPS (navegador)	Mapas base del visor: Mapbox para el mapa principal de kepler.gl y teselas de

Sistema externo	Tipo de interfaz	Descripción de la interacción
		OpenStreetMap para el minimapa del simulador 3D.

Tabla 3: Interfaces del sistema con sistemas externos.

2.1.2. Interfaces con el Usuario

El sistema ofrece dos formas de interacción:

Visor web (interfaz principal): aplicación en el navegador que permite:

1. ver los viajes simulados animados sobre el mapa de Usaquén;
2. parametrizar y lanzar corridas (preset, conteos por modo, semilla, estilo de conducción, cierres viales) y seguir su avance;
3. consultar y configurar intersecciones semaforizadas (ficha, programa, Webster, comparación);
4. ver la priorización de cruces sin semáforo;
5. abrir el simulador 3D comparativo de un semáforo propuesto, con pantalla partida, cámara en tercera persona, HUD de datos y minimapa 2D.

Línea de comandos (CLI): reservada al equipo de desarrollo para ejecutar el pipeline por defecto (`python -m etl.etl`), regenerar las simulaciones precalculadas (`python -m etl.precalculadas`) y levantar la API y el visor.

2.1.3. Interfaces con el Hardware

El pipeline opera sobre una máquina de escritorio o servidor de cómputo convencional. Los requisitos mínimos de hardware están determinados por las fases más intensivas en memoria y cómputo: la construcción de la matriz O/D sobre ~500 000 registros (F2) y la ejecución de netconvert sobre la red vial de Usaquén (F3).

Componente	Especificación mínima
Memoria RAM	8 GB. Requeridos para la carga en memoria del dataset de la encuesta, los polígonos geoespaciales y la red vial OSM simultáneamente durante la fase F2.
Procesador	4 núcleos físicos a $\geq$ 2.0 GHz. netconvert y duarouter utilizan procesamiento paralelo internamente.
Almacenamiento	$\geq$ 10 GB libres. El dump OSM de Bogotá ocupa aproximadamente 2 GB; los artefactos de salida y datos intermedios en requieren hasta 5 GB adicionales.
Red	Acceso a Internet (HTTPS, puerto 443) requerido únicamente durante la descarga inicial de la red OSM. Las fases F2–F5 operan completamente offline.
Sistema Operativo	Ubuntu 22.04, macOS 13+ o Windows 10+. No se requieren controladores especiales ni hardware propietario.

Tabla 4: Requisitos mínimos de hardware.

No se requieren tarjetas gráficas dedicadas, aceleradores GPU, dispositivos de red especiales ni hardware de tiempo real.

2.1.4. Interfaces con el Software

Producto de Software	Versión	Fuente	Propósito en el sistema
Python	3.11	python.org	Lenguaje de implementación del pipeline ETL completo. Requerido por la API TraCI de SUMO para el control de simulaciones.
SUMO	1.19+	sumo.dlr.de	Provee las herramientas netconvert, duarouter y el motor de simulación. Licencia EPL 2.0 (código abierto).
pandas	2.0+	pypi.org	Procesamiento de los datos tabulares de la encuesta de movilidad: lectura XLSX, limpieza, filtrado y construcción de matrices O/D.
geopandas	0.14+	pypi.org	Procesamiento de datos geoespaciales: manejo de polígonos UPZ, asignación de nodos de red a zonas TAZ, validación de cobertura geográfica.
osmium-tool	1.16+	osmcode.org	Extracción del polígono de Usaquén desde el dump OSM de Bogotá mediante recorte geográfico. Invocado como proceso externo desde Python.
pytest	7.0+	pypi.org	Ejecución de pruebas unitarias y de integración del pipeline. Integrado con GitHub Actions para CI automático en cada commit.
GitHub Actions	–	github.com	Integración continua: ejecuta pylint, pytest y verifica portabilidad en Ubuntu y Windows en cada commit a la rama principal.
Git	2.40+	git-scm.com	Control de versiones del código fuente y los documentos del proyecto.
QGIS	3.34+	qgis.org	Validación visual de la red vial generada y de los polígonos TAZ. Usado en la fase de verificación, no en el pipeline automatizado.
Ubuntu 22.04 / macOS 13+ / Windows 10+	–	–	Sistemas operativos soportados. El pipeline debe ejecutarse sin modificaciones en los tres entornos (ver RNF-PORT-001).
FastAPI	0.100+	pypi.org	Backend del módulo web. Expone los resultados de SUMO mediante una API REST consumida por el frontend. Integrado con el pipeline ETL existente.
TraCI / sumolib	1.27.1	pypi.org	Control en vivo de SUMO y lectura de la red desde Python.
React + Vite	18.3 / 6	npmjs.com	Frontend del módulo web (visor `kepler/`).
kepler.gl	3.1	npmjs.com	Mapa animado de viajes y capas de semáforos y priorización.
three.js	0.169.0	npmjs.com	Render 3D del simulador comparativo.
maplibre-gl	4.7.1	npmjs.com	Minimapa 2D del simulador con teselas de OpenStreetMap.

Tabla 5: Interfaces con el software.

Todas las herramientas y bibliotecas listadas son de código abierto o de uso académico gratuito, en cumplimiento de la restricción de licencias del SPMP.

2.1.5. Interfaces de Comunicación

El pipeline de datos opera offline. El sistema completo usa:

- HTTP REST con JSON entre el visor y la API (`localhost:8000`, a través del proxy `/api` de Vite);
- un WebSocket para el simulador 3D;
- TraCI sobre TCP local entre la API y SUMO;
- HTTPS hacia los proveedores de mapas base.

El detalle está en la tabla de interfaces de comunicaciones de la Sección 3.

2.1.6. Restricciones de Memoria

Los requerimientos de memoria del sistema se derivan de la suma de las necesidades de las bibliotecas y procesos que el pipeline invoca simultáneamente:

- **Python 3.11 + pandas + geopandas:** carga del dataset completo de la encuesta (~500,000 registros * 30 columnas) en memoria: aproximadamente 2-3 GB de RAM en el peor caso.
- **osmium-tool / netconvert:** procesamiento de la red vial de Bogotá en formato OSM: aproximadamente 1-2 GB adicionales durante la fase F3.
- **Sistema operativo y procesos base:** reserva de 1-2 GB.
- **Total estimado:** 6-7 GB de RAM en uso simultáneo en el pico de la fase F2/F3, dentro del límite mínimo establecido de 8 GB (ver Tabla 4).

El almacenamiento secundario requerido se detalla en la Tabla 4. No hay requerimientos especiales de memoria de video ni caché de red.

2.1.7. Operaciones

El pipeline opera en un único modo, sin distinción de roles de administrador y usuario final:

Ejecución completa: el usuario lanza el pipeline con los parámetros de configuración deseados y el sistema ejecuta las fases F1 a F5 en secuencia, produciendo todos los artefactos de salida. El tiempo estimado de ejecución completa es de hasta 4 horas en la máquina de referencia (ver RD-001).

Ejecución parcial (reanudación): si una ejecución previa fue interrumpida, el usuario puede reanudar desde la última fase completada usando el flag --resume. El sistema carga el checkpoint guardado y continúa sin reprocesar las fases anteriores (ver RF-016).

Ejecución de escenario hipotético: el usuario modifica el archivo `config.yaml` (por ejemplo, ajustando el factor de escala de demanda o el período horario) y relanza el pipeline para generar un conjunto de artefactos distinto sin modificar el código fuente (ver RF-013).

Uso interactivo (v2.0): el usuario abre el visor, carga una simulación (precalculada o propia), lanza nuevas corridas desde el panel, explora intersecciones y abre comparaciones 3D. La API ejecuta una corrida a la vez y como máximo dos comparaciones 3D simultáneas.

No se definen períodos de mantenimiento ni ventanas de inactividad programadas, dado que el sistema es una herramienta de escritorio de uso bajo demanda. El respaldo de datos está delegado al sistema de control de versiones Git y a las copias locales de los datasets de entrada.

2.1.8. Requerimientos de Adaptación del Sitio

El pipeline no requiere adaptaciones de infraestructura de red ni de hardware en el sitio de uso. Para adaptar el sistema a una localidad distinta de Usaquén (uso futuro), el usuario debe:

- Actualizar el polígono geográfico de recorte en el archivo `config.yaml`.
- Proveer los archivos de aforos HMD para la nueva localidad.
- Verificar que las UPZ de la nueva localidad estén disponibles en la fuente de datos geoespaciales oficial.

No se requieren cambios en el código fuente para la adaptación geográfica, lo cual es un objetivo de diseño explícito del sistema.

2.2. Funciones del Producto

El pipeline está organizado en cinco funcionalidades principales (F1–F5) que corresponden a las fases del proceso ETL. A continuación se describe cada una en lenguaje de usuario. Los casos de uso asociados se documentan en detalle en el documento SDD.

F1 – Ingestión y Validación de Datos: el sistema recibe la Encuesta de Movilidad 2023 en formato XLSX/CSV y los archivos de aforos HMD, valida que contienen las columnas mínimas requeridas, elimina registros duplicados por doble conteo y descarta registros con campos obligatorios vacíos, registrando en un log el porcentaje de datos descartados. También descarga la red vial de Usaquén desde OpenStreetMap si no está disponible localmente.

F2 – Construcción de Matrices Origen-Destino: con los datos limpios y expandidos por los factores muestrales de la encuesta, el sistema construye matrices O/D separadas por modo de transporte (automóvil, bus SITP, motocicleta, bicicleta) y por período horario (AM: 6-9h, PM: 16-19h, nocturno: 21-24h), exportando cada combinación en el formato `.od` que SUMO requiere.

F3 – Generación de Red Vial y Zonas TAZ: el sistema convierte la red OSM de Usaquén al formato `.net.xml` de SUMO usando netconvert, y construye el archivo de zonas de análisis de tráfico (`.taz.xml`) asociando cada UPZ de la localidad con los nodos de la red vial que le corresponden.

F4 – Generación de Archivos de Simulación: el sistema produce el archivo de tipos de vehículo (`.vtype.xml`) con los parámetros físicos estándar de SUMO para cada modo, e invoca duarouter para asignar rutas a los viajes de la matriz O/D, generando el archivo de rutas (`.rou.xml`).

F5 – Variación Paramétrica y Validación: el sistema permite generar múltiples conjuntos de artefactos variando los parámetros del archivo de configuración sin modificar el código, habilitando la exploración de escenarios hipotéticos. Una vez ejecutada la simulación en SUMO, el sistema compara los volúmenes simulados con los aforos reales HMD calculando el estadístico GEH para cada punto de aforo y exportando el resultado en un archivo CSV.

F6 – Módulo Web: el sistema dispone de una interfaz web que permite a los usuarios visualizar de forma interactiva los resultados producidos por SUMO (métricas de congestión, tiempos de viaje, flujos por segmento y niveles de servicio) y configurar nuevos escenarios de simulación desde el navegador, especificando parámetros como el período horario, los modos de transporte activos y el factor de escala de demanda.

F7 – Validación y Reportes: comparación de los volúmenes simulados contra los aforos reales (GEH) y registro de la ejecución.

F8 – Intersecciones y Semaforización:
- consultar cada intersección semaforizada (accesos, volúmenes y fases);
- recortar la red a una zona alrededor de un cruce;
- editar el programa del semáforo o calcularlo con Webster, y comparar programas;
- ver una priorización de los cruces sin semáforo donde podría valer la pena instalar uno.

F9 – Simulador 3D Comparativo: para un cruce sin semáforo, el sistema prepara el escenario actual y una variante con un semáforo propuesto y los simula a la par, mostrándolos lado a lado en

3D con la red, los edificios y los actores reales de la zona. El usuario sigue en tercera persona a un vehículo o peatón y ve su velocidad, su riesgo, la acción que toma y las decisiones posibles en el próximo cruce. Un minimapa 2D lo ubica en la ciudad.

Funcionalidad	Casos de uso	Artefactos producidos
F1 – Ingestión	CU-01	`encuesta_limpia.parquet`, `pipeline.log`
F2 – Matrices O/D	CU-02	`od_auto_AM.od`, `od_bus_PM.od`, … (N modos * 3 períodos)
F3 – Red y TAZ	CU-03	`usaquen.net.xml`, `usaquen.taz.xml`
F4 – Simulación	CU-04	`vehiculos.vtype.xml`, `rutas.rou.xml`
F5 – Escenarios/Val.	CU-05, CU-06	`validacion_geh.csv`, conjuntos de artefactos por escenario
F6 – Módulo Web	CU-07, CU-08	Interfaz web de visualización de resultados SUMO y formulario de parametrización de nuevas simulaciones
F8 – Intersecciones	CU-09 a CU-11	Ficha de intersección, programas `programa_<tl>_<id>.xml`, zonas `runs/zona-*`, tabla de comparación, capa de priorización
F9 – Simulador 3D	CU-12	`runs/sem-*` (zona, variante con semáforo, demanda), frames en vivo por WebSocket, vista 3D partida con HUD y minimapa

Tabla 6: Funciones del producto y artefactos generados.

2.3. Características del Usuario

Se identifican dos clases de usuarios que interactuarán con el pipeline en la versión 1.0:

Clase de usuario	Nivel de privilegios	Rol	Frecuencia de uso	Perfil técnico
Investigador/ Analista de datos	Usuario estándar	Ejecuta el pipeline con distintas configuraciones para generar escenarios de movilidad y analizar los resultados de simulación.	Ocasional (por campaña de análisis, no diaria)	Conocimiento de Python a nivel de usuario (puede editar `config.yaml` y ejecutar comandos en terminal). No requiere conocimiento profundo del código fuente. Familiaridad con conceptos de movilidad urbana y matrices O/D.
Desarrollador del equipo	Desarrollador	Implementa, prueba y mantiene el pipeline. Ejecuta el sistema durante el desarrollo para verificar la correcta generación de artefactos.	Frecuente (diaria durante el desarrollo; semanal durante mantenimiento)	Dominio de Python 3.11, pandas, geopandas, SUMO y Git. Capacidad de modificar el código fuente, escribir pruebas unitarias con pytest y depurar errores del pipeline.
Planificador / Analista de movilidad (no técnico)	Usuario estándar	Consume los resultados de simulación a través del módulo web y configura nuevos escenarios sin	Ocasional (por campaña de análisis)	Sin conocimiento de Python ni de SUMO. Familiaridad con conceptos de movilidad urbana. Accede al sistema

Clase de usuario	Nivel de privilegios	Rol	Frecuencia de uso	Perfil técnico
		necesidad de usar la línea de comandos.		únicamente a través de la interfaz web.

Tabla 7: Características de las clases de usuario.

2.4. Restricciones

2.4.1. Restricciones Generales

- El alcance geográfico del sistema está limitado a la localidad de Usaquén de Bogotá para la versión 1.0. El diseño debe facilitar la extensión a otras localidades sin modificar el código fuente (solo `config.yaml`).
- El pipeline es un proceso monousuario, secuencial y de línea de comandos. No soporta ejecuciones concurrentes sobre los mismos datos de entrada.
- El idioma de los mensajes de log, comentarios del código y documentación es el español. Las variables y funciones del código siguen nomenclatura en inglés (PEP 8).
- El sistema no implementa tolerancia a fallos de hardware (corte de energía, fallo de disco). La recuperación ante estos eventos se limita al mecanismo de checkpoint (RF-016).

2.4.2. Restricciones de Software

- **Lenguaje de programación:** Python 3.11 exclusivamente en el pipeline principal.
- **Licencias:** todas las bibliotecas deben ser de código abierto o uso académico gratuito. No se permiten licencias comerciales (restricción del SPMP).
- **Herramientas SUMO:** la generación de la red y las rutas debe realizarse con netconvert y duarouter. No se reimplementan estas funcionalidades en Python.
- **Sin base de datos relacional:** el almacenamiento intermedio es por archivos en disco (Parquet, CSV, XML). No se usa PostgreSQL, SQLite ni ningún motor de BD.
- **Interfaz web:** el frontend es una aplicación React; requiere un navegador con WebGL 2 para el simulador 3D.

2.4.3. Restricciones de Hardware

- La máquina de ejecución debe cumplir los requisitos mínimos definidos en la Tabla 4 (8 GB RAM, 4 núcleos, $\geq$ 10 GB libres).
- El pipeline no requiere GPU ni hardware especializado.
- El tiempo máximo de ejecución completa es de 4 horas en la máquina de referencia (RD-001). En máquinas que no cumplan los mínimos, el tiempo puede ser mayor y el sistema no garantiza el requerimiento de desempeño.

2.5. Modelo del Dominio

El modelo del dominio del sistema representa las entidades del mundo real que el sistema maneja, transforma y produce. No es un modelo de clases de software sino una representación conceptual del dominio de movilidad urbana y simulación de tráfico.

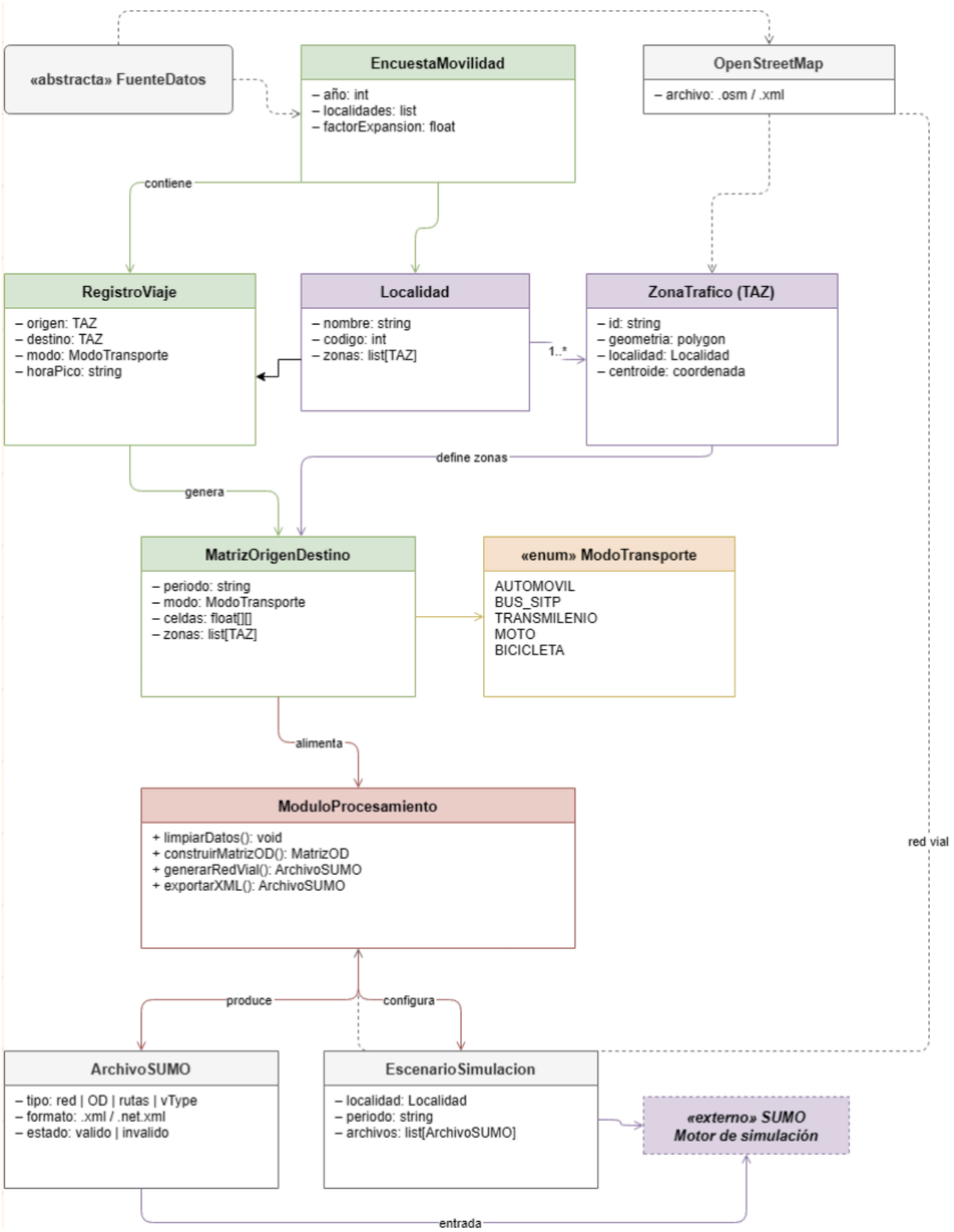


Figura 1: Gráfico modelo de dominio.

Las entidades centrales del dominio y sus relaciones son:

Entidad	Descripción	Fuente / Artefacto
Viaje	Desplazamiento de un hogar entre una zona de origen y una de destino, registrado en la encuesta	Encuesta 2023 (XLSX)

Entidad	Descripción	Fuente / Artefacto
	con su modo de transporte, hora de inicio y factor de expansión muestral.	
Zona TAZ / UPZ	Unidad geográfica de análisis de tráfico. En este proyecto corresponde a las UPZ de Usaquén. Conecta los viajes de la encuesta con los nodos de la red vial.	Shapefile oficial de UPZ
Matriz O/D	Estructura que agrega los viajes expandidos entre pares de zonas TAZ, diferenciada por modo y período horario. Es el principal insumo de demanda para SUMO.	`.od` (generado)
Red Vial	Conjunto de nodos (intersecciones) y aristas (segmentos de vía) de Usaquén, con atributos de velocidad, prioridad y tipo de carretera.	OSM → `.net.xml`
Tipo de Vehículo	Modelo paramétrico que describe el comportamiento físico de un modo de transporte en la simulación: velocidad máxima, aceleración, deceleración y longitud.	`.vtype.xml` (generado)
Ruta	Secuencia de aristas de la red vial asignada por duarouter a un viaje de la matriz O/D para representar el trayecto dentro de la simulación.	`.rou.xml` (generado)
Punto de Aforo	Ubicación física en Usaquén donde la Secretaría de Movilidad registra conteos vehiculares reales en Hora de Máxima Demanda. Usado para validar la simulación.	Base HMD
Escenario de Simulación	Conjunto completo de artefactos SUMO (red, TAZ, rutas, tipos de vehículo) que define una configuración específica de la simulación. Distintos escenarios se generan variando los parámetros del pipeline.	Conjunto de archivos generados
Intersección semaforizada	Nodo de la red controlado por un `tlLogic`, con accesos, conexiones y un programa de fases.	`.net.xml`
Cruce candidato	Nodo sin semáforo con puntaje de priorización; posible ubicación de un semáforo nuevo.	Priorización (RF-032)

Entidad	Descripción	Fuente / Artefacto
Escenario comparativo	Par de zonas idénticas (sin y con semáforo propuesto) con la misma demanda, simuladas en lockstep.	`runs/sem-*`
Actor seguido (ego)	Vehículo o peatón que el usuario observa en tercera persona; tiene telemetría e índice de riesgo por lado.	Frames del WebSocket

Tabla 8: Entidades del modelo del dominio.

Las relaciones clave entre entidades son: un **Viaje** pertenece a una **Zona TAZ** de origen y una de destino; múltiples **Viajes** agregados forman una **Matriz O/D**; una **Matriz O/D** es asignada sobre una **Red Vial** para producir **Rutas**; las **Rutas** junto con los **Tipos de Vehículo** conforman un **Escenario de Simulación**; los resultados del escenario se contrastan con los **Puntos de Aforo** reales para producir la métrica GEH de validación.

2.6. Suposiciones y Dependencias

Las siguientes suposiciones pueden afectar los requerimientos si resultan ser incorrectas:

- La Encuesta de Movilidad de Bogotá 2023 y la base de datos de aforos HMD son accesibles y tienen calidad suficiente para construir matrices O/D representativas de Usaquén. Si la cobertura de UPZ en la encuesta fuera muy baja para Usaquén, los volúmenes de la matriz O/D no serían estadísticamente confiables.
- La red vial de OpenStreetMap para Usaquén tiene atributos suficientes (tipo de vía, velocidades, conectividad) para generar una red simulable en SUMO. Si la cobertura fuera deficiente, sería necesario enriquecerla manualmente con QGIS.
- SUMO 1.19 soporta todos los modos de transporte a modelar (automóvil, bus SITP, motocicleta, bicicleta) con los parámetros por defecto. Cambios en la API de SUMO en versiones posteriores podrían requerir ajustes en el pipeline.
- Los cuatro integrantes del equipo están disponibles durante las 16 semanas del proyecto con una dedicación aproximada de 10 horas semanales por persona.
- La máquina de cómputo disponible cumple los requisitos mínimos definidos en la Tabla 4. Si no los cumple, el requerimiento RD-001 (pipeline en ≤ 4 horas) no puede garantizarse.

2.7. Distribución de Requerimientos

Los requerimientos de la Sección 3 se distribuyen entre los módulos del pipeline de la siguiente manera, lo que facilita su localización para verificación y mantenimiento:

Módulo	Requerimientos asociados	Descripción del módulo
`notebooks/F1.x`, `F2.x`	RF-001, RF-002, RF-003, RF-009, RF-010	Exploración, limpieza de la encuesta y los aforos y construcción de la red 3D y las TAZ (artefactos en `data/processed/`).
`etl/sumo/ trips.py`, `vtypes.py`	RF-004, RF-005, RF-006, RF-008, RF-011, RNF-CON-001, RNF-SEG-001	Generación de viajes TAZ ordenados por salida y tipos de vehículo.
`etl/sumo/ demanda.py`	RF-013, RF-021, RF-022	Presets de transferencia modal y conteos por modo.

Módulo	Requerimientos asociados	Descripción del módulo
`etl/sumo/duarouter.py`, `simulation.py`, `proceso.py`	RF-012, RNF-OBS-001	Ruteo, simulación por TraCI, avance y cancelación de procesos.
`etl/sumo/estilos.py`, `cierres.py`	RF-023, RF-024	Estilos de conducción y cierres viales.
`etl/sumo/fcd.py`, `kepler.py`	RF-027	Exportación de las trayectorias para el visor.
`etl/precalculadas.py`	RF-025, RNF-REP-001	Simulaciones precalculadas reproducibles.
`etl/sumo/intersecciones.py`, `programas_tls.py`, `subred.py`, `priorizacion.py`	RF-028 a RF-032	Intersecciones, programas, Webster, recorte de zonas y priorización.
`etl/sumo/semaforo_nuevo.py`, `demanda_zona.py`, `escena3d.py`, `telemetria.py`	RF-033, RF-034, RF-036, RF-037, RF-039	Semáforo propuesto, demanda de la zona, geometría de la escena e indicadores de riesgo.
`api/api.py`, `api/jobs.py`, `api/comparacion3d.py`	RF-015, RF-018, RF-026, RF-031, RF-035, RF-040, RD-003	API REST/WebSocket, gestor de corridas y bucle lockstep de las comparaciones 3D.
`kepler/src/`	RF-017, RF-019, RF-020, RF-027, RF-034 a RF-039, RD-004, RNF-USA-001	Visor web: mapa, paneles y simulador 3D.
Por definir	RF-014, RF-016, RF-041, RF-043	Validación GEH, reanudación por fase, reporte de la comparación 3D y transporte público real.
Pipeline completo	RD-001, RD-002, RNF-PORT-001, RNF-PORT-002	Requerimientos que aplican al sistema como un todo.

Tabla 9: Distribución de requerimientos por módulo (v2.0).

3. Requerimientos Específicos

Esta sección es en cierto sentido la más importante del proyecto en cuanto a diseño e implementación se refiere. La idea principal de todo el documento, es la de realizar una conexión, un vínculo entre el cliente final y los desarrolladores de la aplicación o el sistema y esta sección es la encargada de esa comunicación.

La especificación de requerimientos busca trasladar las necesidades reales del usuario a un lenguaje más técnico que facilite su construcción, sin embargo es importante mantener un cierto nivel de descripción y lenguaje, para que el cliente no se pierda en tecnicismos y de esta forma hacerlo parte activa de todo el proceso (diseño y construcción).

Para hacer de este documento una parte valiosa en el proceso de construcción del sistema, todos los requerimientos que se ubiquen en esta sección deben cumplir con las siguientes características:

- **Atómico:** cada requerimiento tiene un único propósito y su funcionalidad no puede dividirse en más de una parte.
- **Correcto:** solo se incluyen los requerimientos necesarios según el análisis previo y las entrevistas con los stakeholders.
- **No Ambiguo:** cada requerimiento tiene una sola interpretación válida, independientemente de quién lo lea.
- **Completo:** incluye toda la información necesaria para su perfecto entendimiento: descripción, razón y criterio de medición.
- **Consistente:** ningún requerimiento contradice a otro ni a ninguna sección de los documentos del proyecto.
- **Importancia:** clasificados por prioridad (Alta / Media / Baja) para orientar al equipo hacia las funcionalidades más críticas.
- **Verificable:** cada requerimiento posee un criterio de medición objetivo que permite comprobar su cumplimiento mediante inspección, análisis, demostración o prueba.
- **Trazable:** el origen de cada requerimiento es claro y su relación con otros requerimientos y casos de uso es fácilmente mapeable.
- **Asociado a Versión:** cada requerimiento indica la versión del producto a la que pertenece.
- **No Redundante:** ningún requerimiento se repite ni aparece en más de una parte del documento.
- **Preciso:** se incluyen valores numéricos, umbrales y formatos concretos donde aplique.

3.1. Requerimientos de Interfaces Externas

3.1.1. Interfaces con el Usuario

En la versión 2.0 el usuario final interactúa con el sistema a través del **visor web** (`kepler/`), una aplicación React servida por Vite que consume la API de `simulation-service`. La línea de comandos queda para el equipo de desarrollo (pipeline por defecto y precalculadas). La interfaz está en español y se organiza así:

- **Mapa principal (kepler.gl):** anima los viajes simulados por modo con una capa de trayectorias, una línea de tiempo y una leyenda de modos en la parte superior. La leyenda tiene además los interruptores de las capas **Semáforos** (intersecciones semaforizadas) y **Priorización** (cruces sin semáforo puntuados).
- **Panel derecho con pestañas:**
  - **Demanda:** preset, conteos por modo, escala, semilla y estilo de conducción. Muestra el historial de corridas y las simulaciones precalculadas.
  - **Intersección:** ficha del semáforo con accesos, volúmenes y fases, edición del programa, cálculo de Webster y comparación de programas.
  - **Cierres:** lista de vías cerradas para la próxima corrida.
  - **Semaforizar:** elige un cruce candidato, ajusta el tráfico de fondo y abre el simulador 3D.
- **Tarjeta de actividad:** barras de progreso de la corrida en curso o de la descarga, con botón de cancelar.
- **Simulador 3D comparativo (pantalla completa):**
  - **Vistas:** dos vistas 3D lado a lado, «Sin semáforo (actual)» y «Con semáforo propuesto», y cámara en tercera persona trasera, aérea o libre.
  - **HUD por lado:** velocímetro, índice de riesgo con historial, acción del conductor, decisiones en el próximo cruce y métricas del cruce.
  - **Minimapa:** mapa 2D tipo Maps en la esquina superior derecha.

- **Panel de actores y vías:** para lanzar o seguir actores.

La interfaz se ajusta al tamaño de la ventana; en pantallas de menos de 900 px de ancho las dos vistas 3D se apilan verticalmente. Se usa con ratón y teclado; la cámara libre admite arrastre y rueda.

Comando (desarrollo)	Uso
`python -m etl.etl`	Pipeline completo del escenario por defecto: TSV → trips → duarouter → SUMO → GeoJSON/ trips de kepler.
`python -m etl.precalculadas [preset ...]`	Simula los presets con sus conteos por defecto y guarda los resultados versionados en `precalculadas/`.
`uvicorn api.api:app`	Levanta la API en el puerto 8000 (documentación interactiva en `/docs`).
`npm run dev`	Levanta el visor web en el puerto 5173 con proxy `/api` → `localhost:8000` (HTTP y WebSocket).

Tabla 10: Comandos de línea de comandos del sistema.

3.1.2. Interfaces con el Hardware

El pipeline opera en una máquina local o servidor de cómputo con los siguientes requisitos mínimos de hardware, derivados de los supuestos definidos en el SPMP:

Componente	Requisito mínimo
RAM	8 GB
CPU	4 núcleos físicos a $\geq$ 2.0 GHz
Almacenamiento	$\geq$ 10 GB libres para datasets de entrada, red OSM y archivos de salida
Red	Acceso a Internet para la descarga de la red vial desde OpenStreetMap (descarga única; la ejecución posterior es offline)

Tabla 11: Requisitos mínimos de hardware.

No se requieren dispositivos especiales, aceleradores GPU ni interfaces de red de alta velocidad.

3.1.3. Interfaces con el Software

Software	Versión mínima	Rol en el sistema
Python	3.11	Lenguaje del pipeline ETL, la API y el control de SUMO
SUMO (`eclipse-sumo`)	1.27	Motor de simulación; netconvert, duarouter y las herramientas `cutRoutes.py` y `randomTrips.py`
TraCI / sumolib	1.27.1	Control de SUMO en vivo (avance de corridas, simulador 3D) y lectura de la red
FastAPI + Uvicorn	0.110+	API REST y WebSocket del sistema
pyproj	3.6+	Conversión entre coordenadas de la red y lon/ lat
pandas / geopandas	2.0 / 0.14	Exploración y preparación de datos (notebooks)

Software	Versión mínima	Rol en el sistema
Node.js + Vite	20+ / 6	Servidor de desarrollo y empaquetado del visor web
React	18.3	Interfaz del visor web
kepler.gl	3.1	Mapa con la animación de viajes y las capas de semáforos y priorización
three.js	0.169.0	Render 3D del simulador comparativo
maplibre-gl	4.7.1	Minimapa 2D del simulador (teselas de OpenStreetMap)
Typst	0.12+	Compilación de los documentos SRS, SPMP y VFP
Ubuntu / macOS / Windows	22.04 / 13 / 10	Sistemas operativos soportados
Git	2.40	Control de versiones del código fuente
pytest	7.0	Ejecución de pruebas unitarias e integración
GitHub Actions	–	Integración continua (CI) en cada commit

Tabla 12: Interfaces de software del sistema.

3.1.4. Interfaces de Comunicaciones

El sistema usa las siguientes comunicaciones, todas en la máquina local salvo las teselas de mapa:

Canal	Protocolo / puerto	Uso
Visor ⟶ API	HTTP REST, JSON (`localhost:8000`, vía proxy `/api` de Vite)	Demanda, corridas, intersecciones, programas, comparaciones, priorización y preparación del simulador 3D. Los trips precalculados se sirven en gzip con `Content-Encoding: gzip`.
Visor ⟶ API	WebSocket (`/comparacion3d/{id}`)	Frames del simulador 3D (≈5 por segundo) y comandos del usuario (pausa, velocidad,

Canal	Protocolo / puerto	Uso
		período de riesgo, seguir o inyectar actor).
API → SUMO	TraCI sobre TCP en un puerto libre de `localhost`	Avance de las corridas y control en vivo de los dos SUMO del simulador 3D.
Visor → mapas base	HTTPS 443	Teselas de Mapbox (mapa principal, requiere `VITE_MAPBOX_TOKEN`) y de OpenStreetMap (minimapa).

Tabla 13: Interfaces de comunicaciones.

El pipeline de datos no usa la red; la red vial de OpenStreetMap se descargó una vez durante la preparación de datos.

3.2. Características del Producto de Software (Requerimientos Funcionales)

Los requerimientos funcionales definen las acciones que el sistema debe ejecutar: qué entradas acepta, qué procesamiento realiza y qué salidas produce. Se redactan con la forma "El sistema debe …", y se organizan en cinco funcionalidades que corresponden a las fases del pipeline ETL.

3.2.1. F1 – Ingestión y Validación de Datos de Entrada

N.° Requerimiento	RF-001
Tipo	Funcional
Casos de uso	CU-01
Descripción	El sistema debe leer la base de datos de la Encuesta de Movilidad de Bogotá 2023 en formato XLSX y validar que el archivo contiene al menos las columnas: ID_hogar, modo, origen_UPZ, destino_UPZ y factor_expansión. Si alguna columna obligatoria está ausente, el sistema debe abortar la ejecución y registrar el error en el log indicando el nombre de la columna faltante.
Razón	La encuesta es la fuente primaria de datos de demanda. Sin las columnas mínimas no es posible construir una matriz O/D válida.
Autor	EFSS

Criterio de medición	Dado un archivo XLSX sin la columna destino_UPZ, el sistema debe terminar con código de error distinto de cero y el log debe contener el mensaje “Columna requerida ausente: destino_UPZ”.
Prioridad	Alta
Módulo asociado	`notebooks/F2.x` + `simulation-service/src/etl/sumo/trips.py`
Estado (v2.0)	Parcial. La limpieza y el cruce de la encuesta se hacen en los notebooks de la Fase 2 (`notebooks/F2.x`); el pipeline lee el TSV resultante (`viajes_usaquen_internos.tsv`) y valida sus columnas.
Versión	1.0
Fecha	

Tabla 14: Requerimiento RF-001.

N.° Requerimiento	RF-002
Tipo	Funcional
Casos de uso	CU-01
Descripción	El sistema debe descargar la red vial del polígono de Usaquén desde OpenStreetMap usando osmium-tool, aplicando el recorte geográfico sobre el dump OSM de Bogotá o mediante la API Overpass, y almacenar el resultado en el directorio de datos configurado. Si el archivo ya existe en disco, el sistema debe omitir la descarga y usar la copia local.
Razón	La red OSM es la fuente de infraestructura vial. La descarga debe ser automática para garantizar la reproducibilidad del pipeline sin intervención manual.
Autor	EFSS
Criterio de medición	Ejecutado el pipeline con red local ausente, se debe generar el archivo usaquen.osm.pbf en el directorio de datos. En una segunda ejecución, el log debe indicar “Red OSM encontrada en caché, omitiendo descarga”.
Prioridad	Alta
Módulo asociado	`notebooks/F2.x`, `data/processed/usaquen.osm.xml`
Estado (v2.0)	Parcial. La red se descargó y procesó una vez en los notebooks y se versiona en `data/processed/`; no se descarga en cada ejecución.
Versión	1.0
Fecha	

Tabla 15: Requerimiento RF-002.

N.° Requerimiento	RF-003
Tipo	Funcional
Casos de uso	CU-01
Descripción	El sistema debe eliminar los registros de la encuesta que presenten dobles conteos, aplicando la lógica definida en el documento “Base de datos con factores de ponderación y sin dobles conteos”. Los registros eliminados deben registrarse en el log con su ID_hogar y el motivo de eliminación.
Razón	Los dobles conteos distorsionan los volúmenes de la matriz O/D y producen sobreestimación de la demanda simulada.

Autor	EFSS
Criterio de medición	Dado un conjunto de prueba con 10 registros que incluyan 2 dobles conteos conocidos, el sistema debe producir una tabla de 8 registros y el log debe indicar exactamente 2 registros eliminados por doble conteo.
Prioridad	Alta
Módulo asociado	`notebooks/F2.x`
Estado (v2.0)	Implementado en la preparación de datos (notebooks de la Fase 2).
Versión	1.0
Fecha	

Tabla 16: Requerimiento RF-003.

N.° Requerimiento	RF-004
Tipo	Funcional
Casos de uso	CU-01
Descripción	El sistema debe validar que cada registro de la encuesta tiene valores no nulos y válidos en las columnas modo, origen_UPZ y destino_UPZ. Los registros con al menos uno de esos tres campos vacío o con un valor fuera del dominio definido deben descartarse y registrarse en el log con el ID_hogar y el campo inválido.
Razón	Registros incompletos generan entradas inválidas en la matriz O/D que SUMO no puede procesar.
Autor	EFSS
Criterio de medición	El log debe reportar el porcentaje de registros descartados sobre el total leído. Dado un archivo con 5% de registros con origen_UPZ nulo, el log debe indicar exactamente ese 5% como tasa de descarte.
Prioridad	Alta
Módulo asociado	`simulation-service/src/etl/sumo/trips.py`
Estado (v2.0)	Implementado. Las filas inválidas se descartan y se cuentan en `TripGenerationStats.filas_invalidas`.
Versión	1.0
Fecha	

Tabla 17: Requerimiento RF-004.

N.° Requerimiento	RF-005
Tipo	Funcional
Casos de uso	CU-01
Descripción	El sistema debe aplicar los factores de expansión muestral de la encuesta a cada viaje válido, de manera que el total de viajes expandido sea representativo del universo de viajes de Bogotá y no únicamente de la muestra encuestada.
Razón	Sin expansión, los volúmenes de la matriz O/D reflejan solo la muestra (~23,000 hogares) y no el universo de viajes de la ciudad.
Autor	EFSS

Criterio de medición	La suma de los viajes expandidos debe ser igual a la suma del campo factor_expansión de todos los registros válidos, con una tolerancia de ± 0.01%.
Prioridad	Alta
Módulo asociado	`simulation-service/src/etl/sumo/trips.py`, `configuration.py`
Estado (v2.0)	Implementado. Cada viaje se replica `round(fexp_vj)` veces, con tope `MAX_COPIAS`.
Versión	1.0
Fecha	

Tabla 18: Requerimiento RF-005.

3.2.2. F2 – Construcción de Matrices Origen-Destino

N.° Requerimiento	RF-006
Tipo	Funcional
Casos de uso	CU-02
Descripción	El sistema debe construir matrices O/D diferenciadas por modo de transporte: automóvil particular, transporte público (SITP/TransMilenio), motocicleta y bicicleta, usando los viajes filtrados y expandidos de la encuesta. Cada modo debe producir un archivo .od independiente.
Razón	SUMO modela cada modo de transporte con parámetros de comportamiento distintos (velocidad, aceleración, tamaño del vehículo). Mezclar modos en una sola matriz produce simulaciones incorrectas.
Autor	EFSS
Criterio de medición	Para cada modo con al menos un viaje registrado en la encuesta, el sistema debe generar un archivo .od. El número de archivos generados debe coincidir con el número de modos con viajes en la encuesta.
Prioridad	Alta
Módulo asociado	`simulation-service/src/etl/sumo/trips.py`, `vtypes.py`
Estado (v2.0)	Implementado con un formato distinto: en lugar de un `.od` por modo, cada viaje se emite como `<trip>` TAZ a TAZ con su vType (modo) en un único `.trips.xml`.
Versión	1.0
Fecha	

Tabla 19: Requerimiento RF-006.

N.° Requerimiento	RF-007
Tipo	Funcional
Casos de uso	CU-02
Descripción	El sistema debe construir matrices O/D diferenciadas por al menos tres períodos horarios: AM (6:00–9:00), PM (16:00–19:00) y nocturno (21:00–24:00), usando como criterio de clasificación la hora de inicio del viaje registrada en la encuesta. Cada período debe producir archivos .od separados por modo.

Razón	Los patrones de movilidad varían significativamente por franja horaria. Las horas de máxima demanda (HMD) AM y PM son los períodos de mayor interés.
Autor	EFSS
Criterio de medición	El sistema debe generar al menos 3 * N archivos .od, donde N es el número de modos con viajes. Los archivos de período AM deben contener únicamente viajes con hora de inicio entre 6:00 y 8:59.
Prioridad	Alta
Módulo asociado	`simulation-service/src/etl/sumo/configuration.py`
Estado (v2.0)	Parcial. La versión actual simula la ventana AM de 7:00 a 10:00 (`HORA_INI`, `HORA_FIN`); PM y nocturno quedan pendientes.
Versión	1.0
Fecha	

Tabla 20: Requerimiento RF-007.

N.° Requerimiento	RF-008
Tipo	Funcional
Casos de uso	CU-02
Descripción	El sistema debe exportar cada matriz O/D al formato de archivo .od requerido por SUMO (TAZ-based OD matrix), con la estructura de encabezado, declaración de zonas TAZ y valores de viajes que especifica la documentación oficial de SUMO (versión 1.19+).
Razón	duarouter solo acepta archivos .od con la estructura exacta documentada por SUMO. Un formato incorrecto impide la generación de rutas.
Autor	EFSS
Criterio de medición	Cada archivo .od generado debe ser procesado exitosamente por duarouter sin errores de formato. Se verificará ejecutando duarouter con el archivo generado sobre la red .net.xml de Usaquén.
Prioridad	Alta
Módulo asociado	`simulation-service/src/etl/sumo/trips.py`
Estado (v2.0)	Reemplazado por viajes TAZ (`fromTaz`/`toTaz`) que duarouter rutea directamente.
Versión	1.0
Fecha	

Tabla 21: Requerimiento RF-008.

3.2.3. F3 – Generación de Red Vial y Zonas TAZ

N.° Requerimiento	RF-009
Tipo	Funcional
Casos de uso	CU-03
Descripción	El sistema debe generar un archivo de red vial .net.xml válido para el polígono de Usaquén invocando netconvert sobre la red OSM descargada,

	incluyendo velocidades por tipo de vía, prioridades de paso y tipos de carretera definidos en el archivo de configuración.
Razón	El archivo .net.xml es el insumo de infraestructura obligatorio de SUMO. Sin él no es posible ejecutar ninguna simulación.
Autor	EFSS
Criterio de medición	El archivo .net.xml generado debe superar la validación de netconvert (salida sin errores) y SUMO debe poder cargar la red sin advertencias críticas (tipo ERROR en el log de SUMO).
Prioridad	Alta
Módulo asociado	`notebooks/F2.x`, `data/processed/`
Estado (v2.0)	Implementado. La red 3D (`usaquen_3d_v2.net.xml`, con elevación) se generó con netconvert en los notebooks.
Versión	1.0
Fecha	

Tabla 22: Requerimiento RF-009.

N.° Requerimiento	RF-010
Tipo	Funcional
Casos de uso	CU-03
Descripción	El sistema debe generar el archivo de zonas TAZ (.taz.xml) para Usaquén usando los polígonos de las Unidades de Planeamiento Zonal (UPZ) de la localidad, asignando a cada TAZ los nodos de la red .net.xml que se encuentren dentro de su polígono.
Razón	Las zonas TAZ son la unidad espacial que conecta la matriz O/D con la red vial. Sin ellas duarouter no puede asignar rutas a los viajes.
Autor	EFSS
Criterio de medición	Cada UPZ de Usaquén debe aparecer como una zona en el .taz.xml y debe contener al menos un nodo de la red. El archivo debe ser validado por SUMO sin errores de zona vacía.
Prioridad	Alta
Módulo asociado	`data/processed/usaquen.taz.xml`
Estado (v2.0)	Implementado (`usaquen.taz.xml`).
Versión	1.0
Fecha	

Tabla 23: Requerimiento RF-010.

3.2.4. F4 – Generación de Archivos de Simulación

N.° Requerimiento	RF-011
Tipo	Funcional
Casos de uso	CU-04
Descripción	El sistema debe generar el archivo de tipos de vehículo (.vtype.xml) con los parámetros de comportamiento estándar de SUMO para cada modo de transporte modelado: automóvil particular, bus SITP, motocicleta y

	bicicleta. Los valores de velocidad máxima, aceleración, deceleración y longitud del vehículo deben tomarse de los valores por defecto documentados en SUMO 1.19.
Razón	SUMO requiere que cada vehículo esté asociado a un tipo con parámetros físicos válidos. Sin este archivo la simulación no puede iniciarse.
Autor	EFSS
Criterio de medición	El archivo .vtype.xml debe contener al menos cuatro entradas (una por modo). SUMO debe cargar el archivo sin errores ni advertencias de tipo desconocido.
Prioridad	Alta
Módulo asociado	`simulation-service/src/etl/sumo/vtypes.py`
Estado (v2.0)	Implementado. Diez vTypes (auto, bici, moto, taxi, escolar, especial, informal, otro y bus_pt).
Versión	1.0
Fecha	

Tabla 24: Requerimiento RF-011.

N.° Requerimiento	RF-012
Tipo	Funcional
Casos de uso	CU-04
Descripción	El sistema debe invocar duarouter con la matriz O/D, la red .net.xml y las zonas .taz.xml para generar el archivo de rutas (.rou.xml), configurando el número máximo de iteraciones de asignación dinámica de rutas (DUE) según el parámetro definido en el archivo de configuración.
Razón	El .rou.xml especifica los viajes y sus rutas dentro de la red vial. Es el insumo de demanda que SUMO requiere para ejecutar la simulación.
Autor	EFSS
Criterio de medición	duarouter debe completar la ejecución sin errores y producir un .rou.xml con al menos un viaje por cada par O/D con valor mayor a cero en la matriz .od.
Prioridad	Alta
Módulo asociado	`simulation-service/src/etl/sumo/duarouter.py`
Estado (v2.0)	Implementado (`--ignore-errors --repair`, viajes ordenados por `depart`).
Versión	1.0
Fecha	

Tabla 25: Requerimiento RF-012.

3.2.5. F5 – Variación Paramétrica y Escenarios Hipotéticos

N.° Requerimiento	RF-013
Tipo	Funcional
Casos de uso	CU-05

Descripción	El sistema debe permitir la ejecución del pipeline con distintos valores de los parámetros de entrada (período horario, modo de transporte, localidad, factor de escala de demanda) especificados en el archivo de configuración YAML, generando conjuntos de artefactos SUMO independientes por cada combinación de parámetros sin necesidad de modificar el código fuente.
Razón	La capacidad de generar escenarios hipotéticos (ej. aumento del 20% en demanda vehicular, cierre de vías) es el objetivo diferenciador del sistema respecto a un pipeline ETL convencional.
Autor	EFSS
Criterio de medición	Dados dos archivos de configuración con factores de escala 1.0 y 1.2 respectivamente, el sistema debe generar dos conjuntos de artefactos cuyo total de viajes expandidos difiera exactamente en un 20%.
Prioridad	Alta
Módulo asociado	`simulation-service/src/etl/sumo/demanda.py`, `api/api.py`
Estado (v2.0)	Implementado y ampliado: la variación paramétrica se hace desde la API/visor con presets de demanda, conteos por modo, semilla, estilo de conducción y cierres viales (ver RF-021 a RF-025).
Versión	1.0
Fecha	

Tabla 26: Requerimiento RF-013.

N.° Requerimiento	RF-021
Tipo	Funcional
Casos de uso	CU-05
Descripción	El sistema debe ofrecer presets de demanda que redistribuyan los viajes entre modos sin cambiar el total. Los presets son `encuesta`, `mas_vehiculos`, `mas_peatones` y `dia_sin_carro`. La redistribución se hace con reglas de transferencia modal basadas en P(modo | duración del viaje), calculada sobre la encuesta del día completo ponderada por `fexp`. Un viaje transferido conserva su origen, su destino y su hora de salida.
Razón	Permite evaluar políticas de movilidad (p. ej. día sin carro) con una demanda coherente con los patrones de la encuesta.
Autor	EFSS
Criterio de medición	Con `encuesta` y sus conteos por defecto, los viajes generados son idénticos a la expansión simple de la encuesta (prueba `test_demanda.py`). Con cualquier preset, el total de viajes es igual al de la encuesta.
Prioridad	Alta
Módulo asociado	`simulation-service/src/etl/sumo/demanda.py`
Estado (v2.0)	Implementado.
Versión	2.0
Fecha	

Tabla 27: Requerimiento RF-021.

N.° Requerimiento	RF-022
Tipo	Funcional
Casos de uso	CU-05
Descripción	El sistema debe permitir fijar el número de viajes por modo y la semilla aleatoria de una corrida. Los conteos se asignan entre los viajes de la encuesta de ese modo por el método de mayor residuo. Si el preset no tiene viajes de un modo, la asignación usa el conjunto base de la encuesta.
Razón	El analista necesita escenarios con cantidades exactas por modo y resultados repetibles.
Autor	EFSS
Criterio de medición	Dos corridas con los mismos conteos y semilla generan el mismo `.trips.xml`; el total por modo coincide con el pedido. La API rechaza totales mayores a `MAX_VIAJES` (100 000).
Prioridad	Alta
Módulo asociado	`simulation-service/src/etl/sumo/demanda.py`, `api/api.py`
Estado (v2.0)	Implementado.
Versión	2.0
Fecha	

Tabla 28: Requerimiento RF-022.

N.° Requerimiento	RF-023
Tipo	Funcional
Casos de uso	CU-05
Descripción	El sistema debe permitir elegir un estilo de conducción (agresivo, normal o cauteloso) que ajuste `sigma`, `speedFactor`, `tau` y `lcSpeedGain` de los vTypes motorizados. Peatones y bicicletas no se modifican.
Razón	El comportamiento de los conductores bogotanos es heterogéneo; el estilo permite analizar la sensibilidad de los resultados.
Autor	EFSS
Criterio de medición	Con estilo distinto de `normal`, la corrida incluye un `vtypes_estilo.xml` cuyos vTypes motorizados tienen los parámetros del estilo.
Prioridad	Media
Módulo asociado	`simulation-service/src/etl/sumo/estilos.py`
Estado (v2.0)	Implementado.
Versión	2.0
Fecha	

Tabla 29: Requerimiento RF-023.

N.° Requerimiento	RF-024
Tipo	Funcional
Casos de uso	CU-05

Descripción	El sistema debe permitir cerrar vías durante una corrida. Cada cierre genera un `<rerouter>` con `<closingReroute>` sobre el edge correspondiente, para que los vehículos recalculen su ruta.
Razón	Los cierres por obras o eventos son uno de los escenarios hipotéticos centrales de la propuesta.
Autor	EFSS
Criterio de medición	Una corrida con un cierre incluye `cierres.xml` en sus **additional files** y ningún vehículo atraviesa el edge cerrado en la ventana del cierre.
Prioridad	Media
Módulo asociado	`simulation-service/src/etl/sumo/cierres.py`, `kepler/src/components/PanelCierres.jsx`
Estado (v2.0)	Implementado.
Versión	2.0
Fecha	

Tabla 30: Requerimiento RF-024.

N.° Requerimiento	RF-025
Tipo	Funcional
Casos de uso	CU-07
Descripción	El sistema debe incluir una simulación precalculada y versionada de cada preset con sus conteos por defecto. Cada una se guarda como `trips.geojson.gz`, `volumenes.json` y `meta.json`, con el hash de los insumos. La API debe servirlas comprimidas. El visor debe cargarlas como corridas terminadas y abrir la del preset `encuesta` al iniciar.
Razón	Una corrida completa tarda varios minutos; las precalculadas permiten explorar los presets de inmediato.
Autor	EFSS
Criterio de medición	Al abrir el visor sin corridas propias se muestra la simulación `encuesta`. Si el panel coincide con un preset, el botón ofrece «Cargar simulación» en lugar de ejecutar.
Prioridad	Media
Módulo asociado	`simulation-service/src/etl/precalculadas.py`, `kepler/src/App.jsx`
Estado (v2.0)	Implementado.
Versión	2.0
Fecha	

Tabla 31: Requerimiento RF-025.

3.2.6. F6 – Módulo Web: Visualización y Parametrización

N.° Requerimiento	RF-017
Tipo	Funcional
Casos de uso	CU-07
Descripción	El módulo web debe permitir visualizar los resultados producidos por SUMO en una interfaz gráfica accesible desde el navegador, mostrando al

	menos las siguientes métricas por simulación: volumen vehicular por segmento de vía, tiempo promedio de viaje, índice de congestión por zona TAZ y estadístico GEH por punto de aforo. La visualización debe actualizarse al seleccionar una simulación diferente sin recargar la página.
Razón	Los resultados de SUMO se entregan como archivos XML y CSV de difícil interpretación para usuarios no técnicos. La interfaz web elimina esa barrera y hace el sistema accesible a planificadores de movilidad sin conocimiento de SUMO.
Autor	EFSS
Criterio de medición	Dado un conjunto de resultados de simulación válidos, el módulo web debe renderizar las métricas listadas en menos de 3 segundos desde que el usuario selecciona la simulación. Verificable con prueba manual ante el director.
Prioridad	Alta
Módulo asociado	`kepler/src/App.jsx`
Estado (v2.0)	Parcial. El visor anima los viajes por modo y muestra volúmenes por acceso de cada intersección; tiempos de viaje y GEH por punto de aforo quedan pendientes.
Versión	1.0
Fecha	

Tabla 32: Requerimiento RF-017.

N.° Requerimiento	RF-018
Tipo	Funcional
Casos de uso	CU-07
Descripción	El módulo web debe exponer los resultados de SUMO a través de una API REST que devuelva los datos en formato JSON. La API debe soportar al menos los endpoints: GET /simulaciones (lista de simulaciones disponibles), GET /simulaciones/{id}/metricas (métricas de una simulación específica) y GET /simulaciones/{id}/geh (resultados de validación GEH).
Razón	La separación entre backend (API REST) y frontend (interfaz gráfica) garantiza que los resultados sean consumibles por otras herramientas en el futuro, además de por el frontend actual.
Autor	EFSS
Criterio de medición	Los tres endpoints deben responder con código HTTP 200 y contenido JSON válido ante peticiones con IDs de simulación existentes, y con código 404 ante IDs inexistentes.
Prioridad	Alta
Módulo asociado	`simulation-service/src/api/api.py`
Estado (v2.0)	Implementado con los endpoints actuales: `GET /simulaciones`, `GET /simulaciones/{id}`, `GET /simulaciones/{id}/kepler-trips`, `GET /intersecciones[/{id}]`; 404 si no existe.

Versión	1.0
Fecha	

Tabla 33: Requerimiento RF-018.

N.° Requerimiento	RF-019
Tipo	Funcional
Casos de uso	CU-08
Descripción	El módulo web debe permitir al usuario configurar y lanzar una nueva simulación desde la interfaz gráfica, especificando al menos los siguientes parámetros: período horario (AM / PM / nocturno), modos de transporte activos (selección múltiple) y factor de escala de demanda (valor numérico). Al confirmar, el sistema debe invocar el pipeline ETL con los parámetros seleccionados y notificar al usuario cuando la simulación haya finalizado.
Razón	La parametrización desde la web elimina la necesidad de editar el archivo config.yaml manualmente, haciendo el sistema accesible a usuarios sin conocimiento de Python ni de la estructura de archivos del proyecto.
Autor	EFSS
Criterio de medición	El usuario debe poder lanzar una simulación con parámetros distintos a los de la última ejecución sin editar ningún archivo en disco. El sistema debe mostrar un indicador de progreso durante la ejecución y un mensaje de éxito o error al finalizar.
Prioridad	Alta
Módulo asociado	`kepler/src/components/PanelDemanda.jsx`, `api/jobs.py`
Estado (v2.0)	Implementado (pestaña Demanda: preset, conteos por modo, escala, semilla, estilo y cierres).
Versión	1.0
Fecha	

Tabla 34: Requerimiento RF-019.

N.° Requerimiento	RF-020
Tipo	Funcional
Casos de uso	CU-08
Descripción	El módulo web debe validar en el frontend los parámetros ingresados por el usuario antes de enviarlos al backend, verificando que: el factor de escala de demanda sea un número positivo mayor a cero, al menos un modo de transporte esté seleccionado y el período horario tenga un valor válido. Los errores de validación deben mostrarse al usuario de forma clara antes de permitir el envío del formulario.
Razón	La validación en el frontend previene llamadas innecesarias al backend y el lanzamiento de simulaciones con parámetros inválidos que consumirían recursos de cómputo sin producir resultados útiles.
Autor	EFSS

Criterio de medición	Dado un formulario con factor de escala igual a cero, el sistema debe mostrar un mensaje de error visible y no enviar la petición al backend. Verificable mediante prueba manual.
Prioridad	Media
Módulo asociado	`kepler/src/components/PanelDemanda.jsx`, `api/api.py`
Estado (v2.0)	Implementado (validación en el panel y en el modelo Pydantic de la API).
Versión	1.0
Fecha	

Tabla 35: Requerimiento RF-020.

N.° Requerimiento	RF-026
Tipo	Funcional
Casos de uso	CU-08
Descripción	El sistema debe ejecutar una corrida a la vez en segundo plano. Debe informar su avance global y por etapa (generando demanda, ruteando, simulando, exportando) y permitir cancelarla. Al cancelar se detienen los subprocesos y se borran los archivos de la corrida, salvo su estado.
Razón	Las corridas tardan minutos; el usuario necesita saber cuánto falta y poder detener una corrida equivocada.
Autor	EFSS
Criterio de medición	El avance mostrado crece de forma monótona hasta 100%. Tras cancelar, `runs/<id>/` contiene solo `estado.json` con estado `cancelada`.
Prioridad	Alta
Módulo asociado	`simulation-service/src/api/jobs.py`, `etl/sumo/proceso.py`, `kepler/src/hooks.js`
Estado (v2.0)	Implementado.
Versión	2.0
Fecha	

Tabla 36: Requerimiento RF-026.

N.° Requerimiento	RF-027
Tipo	Funcional
Casos de uso	CU-07
Descripción	El visor debe animar en el mapa los viajes simulados con una capa de trayectorias de kepler.gl: una línea por vehículo o persona con coordenadas `[lon, lat, z, t]` y un color por modo. Al reemplazar una corrida no debe recargar la página.
Razón	Ver los viajes moverse en el tiempo es la forma más directa de entender los resultados de SUMO.
Autor	EFSS
Criterio de medición	Al cargar una corrida, la leyenda muestra los modos presentes con su número de viajes y la línea de tiempo reproduce la ventana 7:00–10:00.
Prioridad	Alta

Módulo asociado	`simulation-service/src/etl/sumo/kepler.py`, `kepler/src/App.jsx`
Estado (v2.0)	Implementado.
Versión	2.0
Fecha	

Tabla 37: Requerimiento RF-027.

3.2.7. F7 – Validación y Reportes

N.° Requerimiento	RF-014
Tipo	Funcional
Casos de uso	CU-06
Descripción	El sistema debe comparar los volúmenes vehiculares simulados por SUMO en los puntos de aforo de Usaquén contra los volúmenes reales de la base de datos HMD, calculando el estadístico GEH para cada punto de aforo. El resultado debe exportarse en un archivo CSV con columnas: punto_aforo, volumen_simulado, volumen_real, GEH.
Razón	El GEH es la métrica estándar de calibración de modelos de tráfico. Un GEH < 5 en al menos el 85% de los puntos indica que el modelo es válido.
Autor	EFSS
Criterio de medición	El archivo de validación CSV debe generarse con una fila por punto de aforo HMD disponible en Usaquén. El sistema debe imprimir en el log el porcentaje de puntos con GEH < 5.
Prioridad	Alta
Módulo asociado	Módulo de Validación (etl/validation.py)
Estado (v2.0)	Pendiente (planeado para la Fase 4).
Versión	1.0
Fecha	

Tabla 38: Requerimiento RF-014.

N.° Requerimiento	RF-015
Tipo	Funcional
Casos de uso	CU-06
Descripción	El sistema debe generar un log de ejecución estructurado en formato texto plano (pipeline.log) que registre: inicio y fin de cada fase, número de registros procesados y descartados por fase, porcentaje de descarte acumulado, métricas GEH obtenidas y cualquier advertencia o error ocurrido durante la ejecución.
Razón	El log es el principal mecanismo de observabilidad del pipeline. Permite al usuario detectar problemas de calidad de datos y reproducir la ejecución con los mismos parámetros.
Autor	EFSS
Criterio de medición	Al finalizar una ejecución exitosa, el archivo pipeline.log debe existir y contener al menos una entrada de inicio y una de fin para cada una de las fases F1 a F5, con marca de tiempo.

Prioridad	Media
Módulo asociado	`simulation-service/src/api/jobs.py`
Estado (v2.0)	Parcial. Cada corrida registra su estado y avance en `runs/<id>/estado.json` y el log de SUMO en `output/`.
Versión	1.0
Fecha	

Tabla 39: Requerimiento RF-015.

N.° Requerimiento	RF-016
Tipo	Funcional
Casos de uso	CU-06
Descripción	El sistema debe guardar un archivo de checkpoint al finalizar cada fase del pipeline (F1 a F5), de forma que ante un fallo en una fase posterior sea posible reanudar la ejecución desde la última fase completada sin reprocesar las anteriores. El usuario debe poder activar o desactivar esta función mediante el archivo de configuración.
Razón	El pipeline completo puede tardar hasta 4 horas. Un fallo en la fase de validación no debe obligar a repetir la ingestión ni la construcción de matrices.
Autor	EFSS
Criterio de medición	Simulado un fallo al inicio de la fase F4, al relanzar el pipeline con el flag y opcion --resume, el sistema debe saltar las fases F1-F3 y comenzar desde F4, produciendo artefactos idénticos a una ejecución completa.
Prioridad	Media
Módulo asociado	Módulo transversal (etl/checkpoint.py)
Estado (v2.0)	Pendiente. Las simulaciones precalculadas (RF-025) evitan recalcular los presets, pero no hay reanudación por fase.
Versión	1.0
Fecha	

Tabla 40: Requerimiento RF-016.

3.2.8. F8 – Intersecciones y Semaforización

N.° Requerimiento	RF-028
Tipo	Funcional
Casos de uso	CU-09
Descripción	El sistema debe mostrar en el mapa las intersecciones semaforizadas de la red (una por `tlLogic`). Al seleccionar una, debe abrir su ficha con: • nombre de las vías que se cruzan (tomado de OpenStreetMap), • accesos y su volumen por tipo de vehículo en la corrida cargada, • fases del programa con su duración y estado.
Razón	La intersección es la unidad de análisis de las decisiones de semaforización.
Autor	EFSS

Criterio de medición	`GET /intersecciones` devuelve un punto por `tlLogic` y `GET /intersecciones/{id}` devuelve la ficha (404 si no existe). Los volúmenes cambian al cambiar la corrida cargada.
Prioridad	Alta
Módulo asociado	`simulation-service/src/etl/sumo/intersecciones.py`, `kepler/src/components/FichaInterseccion.jsx`
Estado (v2.0)	Implementado.
Versión	2.0
Fecha	

Tabla 41: Requerimiento RF-028.

N.° Requerimiento	RF-029
Tipo	Funcional
Casos de uso	CU-09
Descripción	El sistema debe recortar la red y la demanda a un radio configurable (50 a 2000 m) alrededor de un cruce. La red se recorta con netconvert (`--keep-edges.in-geo-boundary`) y la demanda con `cutRoutes.py`, tomando como entrada el vehroute de la corrida cuando exista. Así los escenarios locales se simulan en segundos.
Razón	Simular toda Usaquén para evaluar un solo cruce es lento; el recorte permite iterar rápido sobre programas y escenarios.
Autor	EFSS
Criterio de medición	La zona resultante (`zona.net.xml`) conserva los ids de edges y nodos de la red original, y SUMO la carga sin errores.
Prioridad	Alta
Módulo asociado	`simulation-service/src/etl/sumo/subred.py`, `etl/etl.py` (`run_recorte`)
Estado (v2.0)	Implementado. En v2.0 se corrigió el uso de `cutRoutes.py` (`--disconnected-action`), que antes dejaba las zonas sin demanda.
Versión	2.0
Fecha	

Tabla 42: Requerimiento RF-029.

N.° Requerimiento	RF-030
Tipo	Funcional
Casos de uso	CU-10
Descripción	El sistema debe permitir editar las duraciones y estados de las fases de un semáforo y calcular un programa con el método de Webster a partir de los volúmenes por acceso. Antes de guardarlo debe validar que el estado de cada fase tenga un carácter por movimiento, que haya al menos una fase amarilla de 3 s o más si hay verdes, y que el ciclo sea de al menos 15 s.
Razón	Ajustar tiempos es la intervención más común y barata sobre una intersección.

Autor	EFSS
Criterio de medición	Un programa inválido se rechaza con un mensaje que indica la regla incumplida (y la fase, cuando aplica). El ciclo de Webster $C = \frac{1{,}5L+5}{1-Y}$ nunca es menor a 30 s.
Prioridad	Alta
Módulo asociado	`simulation-service/src/etl/sumo/programas_tls.py`, `FichaInterseccion.jsx`
Estado (v2.0)	Implementado.
Versión	2.0
Fecha	

Tabla 43: Requerimiento RF-030.

N.° Requerimiento	RF-031
Tipo	Funcional
Casos de uso	CU-10
Descripción	El sistema debe comparar dos o más programas de semáforo sobre la misma zona y demanda (una corrida de zona por programa), reportar por programa las métricas de la simulación y marcar el mejor en cada métrica. La versión actual reporta teleports, vehículos insertados y personas cargadas; se deben agregar demora media, longitud de cola y espera peatonal por acceso.
Razón	La decisión entre programas debe sustentarse con métricas cuantitativas comparables.
Autor	EFSS
Criterio de medición	`POST /intersecciones/{id}/comparaciones` devuelve un id; al terminar, `GET /comparaciones/{id}` trae una fila por programa con las métricas.
Prioridad	Media
Módulo asociado	`simulation-service/src/api/api.py`, `kepler/src/components/TablaComparacion.jsx`
Estado (v2.0)	Parcial: la comparación funciona con las métricas de `statistics.xml`; demora, cola y espera peatonal están pendientes.
Versión	2.0
Fecha	

Tabla 44: Requerimiento RF-031.

N.° Requerimiento	RF-032
Tipo	Funcional
Casos de uso	CU-11
Descripción	El sistema debe puntuar de 0 a 100 los cruces sin semáforo (`priority` y `right_before_left`) y clasificarlos en prioritario (≥ 60), recomendado, en observación o sin acción. El puntaje suma: • jerarquía del par de vías (40); • carriles y geometría (15);

	• hueco en un corredor semaforizado, es decir, un semáforo entre 300 y 600 m (15); • demanda peatonal de la corrida (15); • siniestralidad (15, reservado hasta contar con datos). Se excluyen los cruces a menos de 300 m de un semáforo. El mapa debe mostrar los cruces prioritarios y recomendados.
Razón	Ayuda a decidir dónde vale la pena estudiar un semáforo nuevo antes de simularlo (ver F9).
Autor	EFSS
Criterio de medición	Ningún cruce listado tiene un semáforo a menos de 300 m (Haversine). El orden por puntaje es estable para una misma corrida.
Prioridad	Media
Módulo asociado	`simulation-service/src/etl/sumo/priorizacion.py`
Estado (v2.0)	Implementado.
Versión	2.0
Fecha	

Tabla 45: Requerimiento RF-032.

3.2.9. F9 – Simulador 3D Comparativo de Semáforo Propuesto

Esta funcionalidad permite ver de forma tangible, como en un videojuego pero con los datos reales de Usaquén, qué pasaría al poner un semáforo en un cruce que hoy no lo tiene. La pantalla se parte en dos:

- a la izquierda, el cruce como está hoy, sin semáforo;
- a la derecha, el mismo cruce con el semáforo propuesto.

Los dos lados corren la misma demanda, con la misma semilla y a la par (lockstep). Se puede seguir a cualquier actor en tercera persona y ver sus datos, su riesgo y sus decisiones.

N.° Requerimiento	RF-033
Tipo	Funcional
Casos de uso	CU-12
Descripción	Dado un cruce sin semáforo, el sistema debe preparar en segundo plano un escenario comparativo con: • el recorte de la zona (350 m) como escenario actual (`zona.net.xml`); • una variante idéntica con un semáforo de tiempos fijos instalado por netconvert (`--tls.set`) en ese nodo (`zona_tls.net.xml`); • una demanda común a ambos lados: los viajes reales de una corrida que pasan por la zona más un tráfico de fondo por modo generado con `randomTrips.py`, con intensidades (veh/h y peatones/h) ajustables por el usuario.
Razón	Recortada a unos cientos de metros, la demanda real de la encuesta es muy escasa para mostrar el efecto de un semáforo. El tráfico de fondo la completa sin perder los viajes reales.
Autor	EFSS
Criterio de medición	`POST /cruces/{nodo}/comparacion3d` responde 202. En menos de 60 s, `GET /comparacion3d/{id}` pasa a estado `lista`, con `fuente_demanda`

	(`real`, `sintetica` o `real+sintetica`) y el número de fases y el ciclo del semáforo propuesto. Si el nodo ya tiene semáforo, el estado es `error` con un mensaje que lo indica.
Prioridad	Alta
Módulo asociado	`simulation-service/src/etl/sumo/semaforo_nuevo.py`, `demanda_zona.py`, `etl/etl.py` (`run_escenario_semaforo`)
Estado (v2.0)	Implementado.
Versión	2.0
Fecha	

Tabla 46: Requerimiento RF-033.

N.° Requerimiento	RF-034
Tipo	Funcional
Casos de uso	CU-12
Descripción	El visor debe mostrar los dos escenarios en una vista 3D partida. Cada lado debe incluir: • la red 3D real de la zona: carriles con su ancho y marcas, cruces, andenes, ciclorrutas y elevación; • los edificios de Microsoft Building Footprints extruidos; • las luces del semáforo propuesto cambiando según su estado en cada paso; • un modelo por actor según su tipo (carro, taxi, moto, bicicleta, bus o peatón), con el movimiento interpolado entre frames.
Razón	Ver el escenario en 3D hace tangible el resultado de la simulación para usuarios no técnicos.
Autor	EFSS
Criterio de medición	Con la comparación abierta, ambos lados muestran los mismos actores en las mismas posiciones mientras sus trayectorias no diverjan, y el lado derecho muestra las luces del semáforo con la fase reportada por TraCI.
Prioridad	Alta
Módulo asociado	`simulation-service/src/api/comparacion3d.py`, `etl/sumo/escena3d.py`, `kepler/src/components/simulador3d/escena.js`
Estado (v2.0)	Implementado.
Versión	2.0
Fecha	

Tabla 47: Requerimiento RF-034.

N.° Requerimiento	RF-035
Tipo	Funcional
Casos de uso	CU-12
Descripción	El usuario debe poder seguir con cámara en tercera persona trasera a cualquier vehículo o peatón presente en ambos escenarios, o lanzar uno nuevo. Para lanzarlo elige el tipo, la vía de llegada, el giro (recto, izquierda, derecha) y el estilo de conducción; el sistema lo inyecta en los

	dos lados a la vez con la misma ruta. También debe ofrecer cámara aérea y libre, con la cámara libre sincronizada entre ambos lados.
Razón	Seguir un mismo actor en los dos escenarios muestra cómo cambia su comportamiento según haya o no semáforo.
Autor	EFSS
Criterio de medición	Tras lanzar un actor, en menos de 2 s aparece en ambos lados con el mismo id y la cámara lo sigue. Si no hay ruta peatonal por el acceso elegido, el visor muestra un aviso sin interrumpir la simulación.
Prioridad	Alta
Módulo asociado	`simulation-service/src/api/comparacion3d.py` (`Lado.inyectar`), `kepler/src/components/simulador3d/Simulador3D.jsx`
Estado (v2.0)	Implementado.
Versión	2.0
Fecha	

Tabla 48: Requerimiento RF-035.

N.° Requerimiento	RF-036
Tipo	Funcional
Casos de uso	CU-12
Descripción	Para el actor seguido, cada lado debe mostrar en pantalla (HUD): • velocidad y límite del carril, aceleración, distancia al vehículo de adelante; • tiempo de espera, tiempo perdido y estado y distancia del próximo semáforo; • la acción que está tomando (acelera, frena, frena por el semáforo, cede el paso, en cola, cambia de carril, camina, espera para cruzar); • las decisiones posibles en el próximo cruce: cada giro con su estado (verde, rojo, vía principal, cede), si hay tráfico en conflicto y cuál tomará según su ruta; • su intención de cambio de carril y el motivo.
Razón	Explica por qué el actor se comporta como lo hace en cada escenario.
Autor	EFSS
Criterio de medición	Los valores del HUD coinciden con los que reporta TraCI en el mismo paso. Un vehículo detenido frente a un rojo muestra «Detenido en el semáforo».
Prioridad	Alta
Módulo asociado	`simulation-service/src/etl/sumo/telemetria.py`, `kepler/src/components/simulador3d/HUD.jsx`
Estado (v2.0)	Implementado.
Versión	2.0
Fecha	

Tabla 49: Requerimiento RF-036.

N.° Requerimiento	RF-037

Tipo	Funcional
Casos de uso	CU-12
Descripción	El sistema debe calcular cada cierto tiempo, configurable entre 1 y 10 s simulados, un índice de riesgo de 0 a 100 para el actor seguido, con nivel bajo, medio o alto y la causa dominante. El índice toma el peor de tres indicadores sustitutos de seguridad: • TTC con el vehículo de adelante (crítico < 1,5 s); • DRAC (crítico > 3,35 m/s²); • PET aproximado con el tráfico transversal en el cruce (crítico < 1,5 s). Para peatones se usa el tiempo hasta que lo alcance el vehículo más cercano. El HUD debe mostrar el historial reciente del índice.
Razón	El riesgo es el argumento principal para instalar un semáforo; medirlo en el tiempo permite comparar ambos escenarios.
Autor	EFSS
Criterio de medición	Con TTC = 1 s el índice es 100 (alto); sin conflictos es 0 (bajo); el período elegido cambia la frecuencia de las muestras del historial (prueba `test_telemetria.py`).
Prioridad	Alta
Módulo asociado	`simulation-service/src/etl/sumo/telemetria.py`
Estado (v2.0)	Implementado.
Versión	2.0
Fecha	

Tabla 50: Requerimiento RF-037.

N.° Requerimiento	RF-038
Tipo	Funcional
Casos de uso	CU-12
Descripción	El simulador debe mostrar en la esquina superior derecha un minimapa 2D tipo Maps con el cruce estudiado, el límite de la zona simulada, la posición del actor seguido y el rastro de su recorrido.
Razón	Ubica al usuario en la ciudad mientras ve la escena en 3D.
Autor	EFSS
Criterio de medición	El punto del actor en el minimapa sigue sus coordenadas lon/lat reportadas por TraCI y el rastro se borra al cambiar de actor.
Prioridad	Media
Módulo asociado	`kepler/src/components/simulador3d/Minimapa.jsx`
Estado (v2.0)	Implementado.
Versión	2.0
Fecha	

Tabla 51: Requerimiento RF-038.

N.° Requerimiento	RF-039
Tipo	Funcional

Casos de uso	CU-12
Descripción	El usuario debe poder elegir cualquiera de las vías con nombre de la zona para llevar la cámara a ella y ver la lista de actores que circulan por esa vía en ese momento, con su tipo y velocidad, para seguir a uno.
Razón	Permite mirar las distintas vías del cruce y elegir qué actor observar.
Autor	EFSS
Criterio de medición	La lista de una vía solo contiene actores cuyo edge actual pertenece a esa vía.
Prioridad	Media
Módulo asociado	`simulation-service/src/etl/sumo/escena3d.py` (vías), `Simulador3D.jsx`
Estado (v2.0)	Implementado.
Versión	2.0
Fecha	

Tabla 52: Requerimiento RF-039.

N.° Requerimiento	RF-040
Tipo	Funcional
Casos de uso	CU-12
Descripción	Cada lado debe mostrar métricas del cruce actualizadas en cada frame: • vehículos a menos de 60 m del cruce y cuántos están detenidos; • espera media; • conflictos (pares de vehículos con PET < 1,5 s); • colisiones detectadas por SUMO; • vehículos y peatones en la zona.
Razón	Resume, lado a lado, el efecto del semáforo propuesto sobre la fluidez y la seguridad.
Autor	EFSS
Criterio de medición	Las métricas de ambos lados se calculan con la misma definición y radio; sin actores en el cruce, los contadores instantáneos son 0.
Prioridad	Alta
Módulo asociado	`simulation-service/src/api/comparacion3d.py` (`Lado.actores`, `Lado.contar_conflictos`)
Estado (v2.0)	Implementado.
Versión	2.0
Fecha	

Tabla 53: Requerimiento RF-040.

N.° Requerimiento	RF-041
Tipo	Funcional
Casos de uso	CU-12
Descripción	El sistema debe exportar un reporte de la comparación 3D (PDF o CSV) con:

	• las métricas acumuladas de cada lado; • la distribución del índice de riesgo; • los datos del semáforo propuesto (fases y ciclo); • un resumen de recomendación.
Razón	El análisis debe poder entregarse a la entidad que toma la decisión.
Autor	EFSS
Criterio de medición	Al cerrar una comparación, el usuario puede descargar el reporte con los valores de ambos lados.
Prioridad	Media
Módulo asociado	Por definir
Estado (v2.0)	Pendiente (por desarrollar).
Versión	2.0
Fecha	

Tabla 54: Requerimiento RF-041.

N.° Requerimiento	RF-042
Tipo	Funcional
Casos de uso	CU-12
Descripción	El simulador debe permitir editar el programa del semáforo propuesto (ciclo y repartos, o Webster con los volúmenes de la zona) y reiniciar la comparación con el nuevo programa.
Razón	El programa por defecto de netconvert no es óptimo; la comparación es más justa con un programa ajustado a la demanda.
Autor	EFSS
Criterio de medición	El ciclo elegido se refleja en `GET /comparacion3d/{id}` y en la fase mostrada en el HUD del lado con semáforo.
Prioridad	Media
Módulo asociado	`semaforo_nuevo.py` (`ciclo_s` ya soportado por la API interna), `programas_tls.py`
Estado (v2.0)	Parcial: el backend acepta `ciclo_s`; falta exponerlo en el visor.
Versión	2.0
Fecha	

Tabla 55: Requerimiento RF-042.

N.° Requerimiento	RF-043
Tipo	Funcional
Casos de uso	CU-05, CU-12
Descripción	El sistema debe modelar el transporte público con buses, paradas y rutas reales (GTFS de TransMilenio/SITP mediante `gtfs2pt`) y viajes de personas con `personTrip modes="public"`, en lugar de representarlo como caminata.
Razón	Hoy los viajes de transporte público se emiten como `<person><walk>`, lo que subestima los buses en las vías y en los cruces.

Autor	EFSS
Criterio de medición	Una corrida con transporte público muestra buses deteniéndose en paradas y personas que los abordan.
Prioridad	Media
Módulo asociado	Por definir
Estado (v2.0)	Pendiente (por desarrollar).
Versión	2.0
Fecha	

Tabla 56: Requerimiento RF-043.

3.3. Requerimientos de Desempeño

Los siguientes requerimientos establecen los umbrales cuantitativos de rendimiento del sistema, medibles durante las fases de verificación y validación.

N.° Requerimiento	RD-001
Tipo	Desempeño – Latencia
Casos de uso	CU-01 a CU-06
Descripción	El pipeline completo (fases F1 a F5) debe completarse en un tiempo total menor o igual a 4 horas en una máquina con 8 GB de RAM y CPU de 4 núcleos físicos, procesando los datos de la localidad de Usaquén.
Razón	El equipo de cómputo disponible para el proyecto tiene exactamente esas especificaciones. Un tiempo mayor haría inviable la exploración iterativa de escenarios hipotéticos en una jornada de trabajo.
Autor	EFSS
Criterio de medición	Ejecutado el pipeline completo en la máquina de referencia con el dataset de Usaquén, el tiempo registrado entre el inicio de F1 y la finalización de F5 debe ser $\leq$ 14,400 segundos.
Prioridad	Alta
Módulo asociado	Pipeline completo
Estado (v2.0)	Cumplido: la corrida completa con la demanda de la encuesta (24.600 viajes) toma varios minutos.
Versión	1.0
Fecha	

Tabla 57: Requerimiento RD-001.

N.° Requerimiento	RD-002
Tipo	Desempeño – Tasa de procesamiento
Casos de uso	CU-02
Descripción	La construcción de la matriz O/D (fase F2) debe completarse en un tiempo máximo de 30 minutos para un conjunto de datos de entrada de aproximadamente 500,000 registros de encuesta.
Razón	La encuesta de movilidad 2023 tiene del orden de 500,000 registros de viaje. Un tiempo mayor en esta fase comprometería el tiempo total del pipeline (RD-001).

Autor	EFSS
Criterio de medición	Ejecutada únicamente la fase F2 sobre el dataset completo de la encuesta en la máquina de referencia, el tiempo de ejecución debe ser $\leq$ 1,800 segundos.
Prioridad	Media
Módulo asociado	Módulo de Construcción O/D (etl/od_builder.py)
Estado (v2.0)	No aplica en la versión actual: la matriz se arma en los notebooks.
Versión	1.0
Fecha	

Tabla 58: Requerimiento RD-002.

N.° Requerimiento	RD-003
Tipo	Desempeño – Latencia
Casos de uso	CU-12
Descripción	La preparación de una comparación 3D (recorte, demanda y semáforo propuesto) debe tardar como máximo 60 s. Durante la comparación, el servidor debe emitir al menos 4 frames por segundo con los dos lados a velocidad 1× para zonas de hasta 200 actores por lado.
Razón	Por debajo de ese ritmo la animación pierde continuidad incluso con interpolación.
Autor	EFSS
Criterio de medición	Medido en la máquina de referencia, la preparación tardó 14 a 40 s y el bucle mantuvo 5 frames/s con unos 60 vehículos y 95 peatones por lado.
Prioridad	Alta
Módulo asociado	`simulation-service/src/api/comparacion3d.py`
Estado (v2.0)	Cumplido en las pruebas de integración.
Versión	2.0
Fecha	

Tabla 59: Requerimiento RD-003.

N.° Requerimiento	RD-004
Tipo	Desempeño – Render
Casos de uso	CU-12
Descripción	El visor debe renderizar las dos escenas 3D a 30 cuadros por segundo o más en un equipo con GPU integrada. Para lograrlo, las geometrías estáticas (carriles, cruces y edificios) se fusionan en pocas mallas por material.
Razón	Una experiencia «tipo videojuego» requiere fluidez visual.
Autor	EFSS
Criterio de medición	Medido con las herramientas de rendimiento del navegador durante 60 s de comparación a velocidad 1×.
Prioridad	Media
Módulo asociado	`kepler/src/components/simulador3d/escena.js`

Estado (v2.0)	Por verificar en la prueba de aceptación.
Versión	2.0
Fecha	

Tabla 60: Requerimiento RD-004.

3.4. Restricciones de Diseño

Las siguientes restricción se identificaron para delimitar el alcance del proyecto y las herramientas tecnológicas a utilizar, sin embargo cabe mencionar que esto puede cambiar ya que, puede haber una tecnología que disminuya el tiempo de implementación.

- **Lenguaje de programación:** Python 3.11. De momento se tiene pensado utilizar Python como lenguaje de programación principal, no obstante lenguajes como C/C++ son utilizados en gran parte para el desarrollo de SUMO, no se descarta el uso de estos dos también para temas de integración o módulos adicionales.
- **Estilo de código:** el código fuente debe cumplir con la guía de estilo PEP 8. El cumplimiento se verifica automáticamente mediante pylint en cada ejecución de CI.
- **Herramientas del ecosistema SUMO:** la generación de la red (.net.xml) y de las rutas (.rou.xml) debe realizarse con netconvert y duarouter respectivamente. No es ideal reimplementar estas funcionalidades.
- **Software de código abierto:** todas las herramientas y bibliotecas usadas deben ser preferiblemente de código abierto o de uso académico gratuito.
- **Arquitectura del pipeline:** el sistema debe seguir una arquitectura de pipeline por fases secuenciales (F1-F5), donde cada fase produce artefactos que sirven como entrada de la fase siguiente.
- **Control de versiones:** todo el código fuente debe versionarse en el repositorio GitHub del proyecto con una estructura de carpetas definida en el SPMP.
- **Arquitectura del módulo web:** el módulo web sigue una arquitectura de dos capas: backend REST/WebSocket en Python (FastAPI, `simulation-service/`) y frontend React en el navegador (`kepler/`). El backend reutiliza los módulos del pipeline (`etl/sumo/`) sin duplicar lógica de procesamiento.
- **Patrón de los pasos del pipeline:** cada paso de `etl/sumo/` define un `*Config` inmutable, un `*Result` y un `*Stats` mutable, y lanza un `*Error(RuntimeError)` propio si falla la herramienta de SUMO que invoca. `configuration.py` es la única fuente de rutas y constantes.
- **Control en vivo:** toda interacción en tiempo real con SUMO (avance de corridas, simulador 3D) se hace por TraCI. No se interpreta la salida de consola de SUMO, porque llega en bloque al terminar el proceso.
- **Render 3D en el cliente:** la escena 3D se construye en el navegador con three.js a partir de la geometría de la red en coordenadas de SUMO; el servidor solo envía posiciones y estados.

Para las restricciones de diseño de arquitectura interna, modelos de datos y diagramas de componentes, se remite al documento SDD (Software Design Document).

3.5. Atributos del Sistema de Software (Requerimientos No Funcionales)

3.5.1. Confiabilidad

N.° Requerimiento	RNF-CON-001
Tipo	No Funcional – Confiabilidad
Casos de uso	CU-01 a CU-06

Descripción	El sistema debe manejar valores faltantes (NaN) en las columnas modo, origen_UPZ y destino_UPZ de la encuesta sin lanzar excepciones no controladas. Los registros afectados deben descartarse silenciosamente y el porcentaje de registros descartados debe registrarse en el log al finalizar la fase F1.
Razón	Los datasets reales de encuesta contienen inevitablemente registros incompletos. Un fallo por NaN detendría el pipeline completo.
Autor	EFSS
Criterio de medición	Dado un archivo de prueba con 10% de registros con NaN en origen_UPZ, el pipeline debe completarse sin excepciones y el log debe reportar exactamente 10% de registros descartados en F1.
Prioridad	Alta
Módulo asociado	`simulation-service/src/etl/sumo/trips.py`
Estado (v2.0)	Implementado (filas inválidas descartadas y contadas).
Versión	1.0
Fecha	

Tabla 61: Requerimiento RNF-CON-001.

N.° Requerimiento	RNF-CON-002
Tipo	No Funcional – Confiabilidad
Casos de uso	CU-01 a CU-06
Descripción	Ante un fallo en cualquier fase del pipeline (excepción no controlada, archivo de entrada no encontrado, timeout de herramienta externa), el sistema debe guardar el estado procesado hasta ese punto en un archivo de checkpoint y terminar con un mensaje de error descriptivo en el log, indicando la fase fallida y la causa.
Razón	El pipeline puede ejecutarse por horas. Perder todo el progreso por un fallo al final de la ejecución es inaceptable para el flujo de trabajo del equipo.
Autor	EFSS
Criterio de medición	Simulado un fallo forzado al inicio de F4, el directorio de salida debe contener los artefactos de F1-F3 y el log debe indicar “Fase F4 fallida: [causa]”.
Prioridad	Alta
Módulo asociado	`simulation-service/src/api/jobs.py`
Estado (v2.0)	Parcial: un fallo deja el error en `estado.json`; cancelar borra la corrida.
Versión	1.0
Fecha	

Tabla 62: Requerimiento RNF-CON-002.

3.5.2. Disponibilidad

N.° Requerimiento	RNF-DISP-001
Tipo	No Funcional – Disponibilidad

Casos de uso	CU-01 a CU-06
Descripción	El pipeline debe poder ejecutarse de forma completamente offline una vez que los datos de entrada (encuesta XLSX, dump OSM, aforos HMD) hayan sido descargados. No debe depender de servicios en línea durante las fases F2 a F5.
Razón	El entorno de cómputo del equipo puede no tener acceso estable a Internet durante la ejecución de simulaciones. La disponibilidad del pipeline no debe depender de servicios externos en las fases críticas.
Autor	EFSS
Criterio de medición	Desconectada la red durante las fases F2-F5, el pipeline debe completarse sin errores de conexión.
Prioridad	Media
Módulo asociado	Pipeline completo
Estado (v2.0)	Cumplido para el pipeline; el minimapa del simulador 3D usa teselas de OpenStreetMap en línea.
Versión	1.0
Fecha	

Tabla 63: Requerimiento RNF-DISP-001.

3.5.3. Seguridad

N.° Requerimiento	RNF-SEG-001
Tipo	No Funcional – Seguridad
Casos de uso	CU-01 a CU-06
Descripción	El sistema no debe exponer los microdatos de la Encuesta de Movilidad 2023 en los artefactos de salida. Los archivos .od, .rou.xml y .taz.xml deben contener únicamente datos agregados por zona TAZ, sin identificadores de hogar ni de persona.
Razón	La Encuesta de Movilidad contiene datos de movilidad de hogares que pueden ser sensibles. Los artefactos de salida son compartidos con el director y los jurados del proyecto.
Autor	EFSS
Criterio de medición	Inspeccionados los archivos de salida, ninguno debe contener el campo ID_hogar ni ningún identificador individual de la encuesta.
Prioridad	Media
Módulo asociado	Módulo de Construcción O/D (etl/od_builder.py)
Estado (v2.0)	Cumplido: los archivos de SUMO solo contienen viajes por TAZ con ids sintéticos (`auto_N`, `peaton_N`).
Versión	1.0
Fecha	

Tabla 64: Requerimiento RNF-SEG-001.

3.5.4. Mantenibilidad

N.° Requerimiento	RNF-MAN-001

Tipo	No Funcional – Mantenibilidad
Casos de uso	–
Descripción	Toda función pública del pipeline debe tener un docstring con al menos los siguientes campos: descripción de la función, parámetros de entrada con su tipo, valor de retorno con su tipo y un ejemplo de uso.
Razón	El sistema será mantenido por cuatro integrantes del equipo con roles distintos. Sin documentación interna, la curva de aprendizaje para modificar una fase es demasiado alta.
Autor	EFSS
Criterio de medición	Ejecutado pydocstyle sobre el paquete etl/, el número de funciones públicas sin docstring completo debe ser cero.
Prioridad	Media
Módulo asociado	Pipeline completo
Estado (v2.0)	Parcial: los módulos tienen docstring de módulo y de las funciones públicas principales.
Versión	1.0
Fecha	

Tabla 65: Requerimiento RNF-MAN-001.

3.5.5. Portabilidad

N.° Requerimiento	RNF-PORT-001
Tipo	No Funcional – Portabilidad
Casos de uso	–
Descripción	El pipeline debe ejecutarse sin modificaciones en los sistemas operativos Ubuntu 22.04, macOS 13+ y Windows 10+, usando Python 3.11 y SUMO 1.19 instalados en el sistema. La instalación de dependencias debe realizarse únicamente con pip install -r requirements.txt.
Razón	Los cuatro integrantes del equipo trabajan en sistemas operativos distintos. El pipeline debe funcionar en todos ellos sin configuraciones especiales.
Autor	EFSS
Criterio de medición	El pipeline de CI de GitHub Actions debe incluir jobs para Ubuntu y Windows. Ambos jobs deben completarse exitosamente en cada commit a la rama principal.
Prioridad	Alta
Módulo asociado	Pipeline completo
Estado (v2.0)	Parcial: probado en Windows 11; el CI solo corre las pruebas de la raíz.
Versión	1.0
Fecha	

Tabla 66: Requerimiento RNF-PORT-001.

N.° Requerimiento	RNF-PORT-002
Tipo	No Funcional – Portabilidad

Casos de uso	–
Descripción	Todas las dependencias de Python del pipeline deben estar especificadas en el archivo requirements.txt con versiones exactas (pinned), de forma que pip install -r requirements.txt produzca un entorno idéntico en cualquier máquina.
Razón	Sin versiones exactas, actualizaciones de bibliotecas como pandas o geopandas pueden romper el pipeline silenciosamente entre ejecuciones de distintos integrantes.
Autor	EFSS
Criterio de medición	Todas las entradas de requirements.txt deben seguir el formato biblioteca==X.Y.Z. La presencia de rangos (≥, ~=) debe ser detectada y reportada por el CI como advertencia.
Prioridad	Media
Módulo asociado	Configuración del entorno
Estado (v2.0)	Parcial: `simulation-service/pyproject.toml` fija `traci==1.27.1`; el resto usa rangos.
Versión	1.0
Fecha	

Tabla 67: Requerimiento RNF-PORT-002.

3.5.6. Reproducibilidad

N.° Requerimiento	RNF-REP-001
Tipo	No Funcional – Reproducibilidad
Casos de uso	CU-01 a CU-06
Descripción	Dado el mismo conjunto de archivos de entrada (encuesta XLSX, dump OSM, aforos HMD) y el mismo archivo de configuración YAML, dos ejecuciones consecutivas del pipeline deben producir artefactos de salida bit-a-bit idénticos, sin importar en qué máquina o en qué momento se ejecuten.
Razón	La reproducibilidad es un requisito fundamental del proyecto. Los resultados publicados en el informe final deben poder ser verificados.
Autor	EFSS
Criterio de medición	Ejecutado el pipeline dos veces con los mismos inputs, el hash SHA-256 de cada archivo de salida debe ser idéntico entre ambas ejecuciones. Verificable con el script scripts/verify_reproducibility.py incluido en el repositorio.
Prioridad	Alta
Módulo asociado	`simulation-service/src/etl/precalculadas.py`
Estado (v2.0)	Implementado para las precalculadas: gzip con `mtime=0` y hash SHA-256 de los insumos en `meta.json`.
Versión	1.0
Fecha	

Tabla 68: Requerimiento RNF-REP-001.

3.5.7. Observabilidad

N.° Requerimiento	RNF-OBS-001
Tipo	No Funcional – Observabilidad
Casos de uso	CU-01 a CU-06
Descripción	El pipeline debe reportar el progreso de cada fase en la consola con una barra de progreso o mensajes de estado al menos cada 30 segundos durante operaciones que tomen más de un minuto, indicando la fase actual, el número de registros procesados y el tiempo transcurrido.
Razón	Una ejecución de hasta 4 horas sin retroalimentación visual hace imposible distinguir si el pipeline está procesando lentamente o si se ha colgado.
Autor	EFSS
Criterio de medición	Durante la ejecución de F2 con el dataset completo, el usuario debe ver al menos un mensaje de progreso en consola por cada minuto de ejecución.
Prioridad	Media
Módulo asociado	`simulation-service/src/api/jobs.py`, `etl/sumo/proceso.py`
Estado (v2.0)	Implementado: avance real por etapa (duarouter por último `depart`, SUMO por TraCI) visible en el visor.
Versión	1.0
Fecha	

Tabla 69: Requerimiento RNF-OBS-001.

3.5.8. Usabilidad

N.° Requerimiento	RNF-USA-001
Tipo	No Funcional – Usabilidad
Casos de uso	CU-07 a CU-12
Descripción	Toda la interacción del usuario final debe hacerse desde el visor web en español, sin editar archivos XML ni usar la línea de comandos. Cada operación de más de 2 s (corrida, descarga, preparación de la comparación 3D) debe mostrar su avance o la etapa en curso, y los errores deben explicarse con un mensaje legible.
Razón	El usuario objetivo (planificador de movilidad) no conoce SUMO ni Python.
Autor	EFSS
Criterio de medición	Prueba de usuario con el director: lanzar una corrida, abrir una intersección y una comparación 3D sin asistencia del equipo.
Prioridad	Alta
Módulo asociado	`kepler/src/`
Estado (v2.0)	Implementado; pendiente la prueba de usuario.
Versión	2.0
Fecha	

Tabla 70: Requerimiento RNF-USA-001.

3.6. Requerimientos de la Base de Datos

El almacenamiento de los datos a manejar tiene el siguiente formatos:

Fase	Formato	Descripción
F1 – Ingestión	Parquet	Datos limpios de la encuesta con factor de expansión aplicado. Permite lectura parcial y compresión eficiente para ~500K registros.
F2 – Matrices O/D	.od	Formato nativo de SUMO para matrices TAZ. Texto plano estructurado según documentación oficial de SUMO 1.19.
F3 – Red vial	.net.xml	Formato XML nativo de SUMO generado por netconvert. Contiene nodos, aristas, velocidades y prioridades.
F3 – Zonas TAZ	.taz.xml	Formato XML nativo de SUMO. Lista de zonas con sus nodos de red asociados.
F4 – Tipos vehículo	.vtype.xml	Formato XML nativo de SUMO. Parámetros físicos y de comportamiento por modo de transporte.
F4 – Rutas	.rou.xml	Formato XML nativo de SUMO generado por duarouter. Contiene viajes y rutas asignadas.
F5 – Validación	CSV	Tabla de resultados GEH por punto de aforo. Columnas: punto_aforo, volumen_simulado, volumen_real, GEH.
Transversal	JSON	Archivos de checkpoint por fase. Contienen el estado de ejecución para permitir reanudación parcial.
Corridas (v2.0)	`runs/<id>/`	Trips, rutas, salidas de SUMO, `estado.json` y trips de kepler de cada corrida lanzada desde la API. No se versiona.

Fase	Formato	Descripción
Precalculadas (v2.0)	`.geojson.gz` + JSON	`precalculadas/<preset>/`: trips comprimidos, volúmenes por edge y metadatos con el hash de los insumos. Se versiona.
Zonas y comparaciones 3D (v2.0)	`.net.xml`, `.rou.xml`, `.trips.xml`	`runs/zona-*` y `runs/sem-*`: red recortada, variante con el semáforo propuesto, demanda real recortada y tráfico de fondo. Se borran al cerrar la comparación.
Cachés (v2.0)	JSON	`cache/semaforos.json` (catálogo de intersecciones) y `volumenes.json` por corrida; se invalidan por fecha y tamaño del archivo de origen.

Tabla 71: Tipos de datos almacenados por el pipeline.

No se requiere indexación, uso de llaves primarias ni manejo de transacciones concurrentes, dado que el pipeline es un proceso secuencial monousuario. Las consultas sobre los datos intermedios se realizan mediante pandas y geopandas directamente sobre los archivos en disco.

4. Proceso de Ingeniería de Requerimientos

Esta sección describe el proceso que el equipo siguió para construir la especificación de requerimientos contenida en este documento. Se presentan las técnicas de elicitación utilizadas, la forma en que los requerimientos fueron analizados y priorizados, el esquema de trazabilidad adoptado y el proceso de control de cambios, el cual es complementario a lo definido en la sección de Control de Requerimientos del SPMP del proyecto.

4.1. Elicitación de Requerimientos

La elicitación es el proceso de descubrir, extraer y comprender las necesidades de los stakeholders del sistema. Para el proyecto, se emplearon tres técnicas complementarias, seleccionadas en función de la naturaleza técnica del dominio y de la disponibilidad de los actores involucrados.

4.1.1. Entrevistas con el Director del Proyecto

Se realizaron sesiones de trabajo estructuradas con el Ing. Andrés Calderón Romero, director del proyecto, quien cumple el rol de cliente técnico del sistema. Estas sesiones se llevaron a cabo al inicio del proyecto (Semana 1) y al cierre de la Fase 1 de CRISP-DM (Semana 3), con el objetivo de:

- Identificar las fuentes de datos disponibles (Encuesta de Movilidad 2023, aforos HMD, red OSM) y sus restricciones de acceso y calidad.
- Delimitar el alcance geográfico del caso de validación (localidad de Usaquén).
- Validar los modos de transporte prioritarios a modelar: automóvil particular, bus SITP/ TransMilenio, motocicleta y bicicleta.
- Acordar los criterios de aceptación del modelo de simulación basados en el estadístico GEH (umbral GEH < 5 en al menos el 85% de los puntos de aforo).

4.1.2. Análisis de Documentación

Dado que el dominio del problema combina ciencia de datos, simulación de tráfico y política pública de movilidad, una parte significativa de los requerimientos se derivó del análisis directo de fuentes documentales:

- **Encuesta de Movilidad de Bogotá 2023:** el análisis de la estructura del dataset (columnas, tipos de dato, dominio de valores, proporción de registros incompletos) fue determinante para definir los requerimientos de validación y limpieza de datos (RF-001 a RF-005).
- **Documentación oficial de SUMO 1.19:** la especificación de los formatos de archivo `.net.xml`, `.taz.xml`, `.od`, `.rou.xml` y `.vtype.xml` fue la fuente directa para los requerimientos de generación de artefactos (RF-008 a RF-012).
- **Base de datos de aforos HMD:** el análisis de los puntos de aforo disponibles en Usaquén definió la granularidad y el período de las matrices O/D requeridas (RF-006, RF-007).
- **TomTom Traffic Index 2025 e informes de la Secretaría Distrital de Movilidad:** usados para contextualizar la importancia del sistema y validar los períodos horarios AM/PM como los más relevantes para el análisis.

4.1.3. Exploración de Datos (CRISP-DM – Fase 1)

Las Fases 1 y 2 del modelo de ciclo de vida del proyecto (comprensión del dominio y preparación de datos, bajo CRISP-DM) generaron hallazgos concretos que se tradujeron directamente en requerimientos.

- La presencia de dobles conteos en la encuesta, que motivó el requerimiento RF-003.
- La variabilidad en la cobertura de UPZ entre los registros de la encuesta y los polígonos oficiales, que motivó el requerimiento RF-010.
- La duración estimada del procesamiento de ~500,000 registros, que fundamentó el requerimiento de desempeño RD-002.
- La necesidad de diferenciar al menos tres períodos horarios (RF-007), dado que la distribución de viajes en la encuesta mostró patrones claramente distintos en AM, PM y nocturno.

4.2. Análisis y Priorización de Requerimientos

Una vez elicitados los requerimientos candidatos, el equipo realizó una sesión de análisis conjunto para depurarlos, verificar su consistencia y asignarles prioridad. Se aplicaron los siguientes criterios:

4.2.1. Resolución de Conflictos

Durante el análisis se identificaron dos conflictos entre requerimientos candidatos:

- **RF-006 vs. RF-007 (versión preliminar):** inicialmente se habían redactado como dos requerimientos separados que definían períodos horarios para automóviles y para los demás modos respectivamente, con definiciones de períodos inconsistentes. Se resolvió unificando la definición de períodos en RF-007 (AM: 6–9h, PM: 16–19h, nocturno: 21–24h) y haciendo que RF-006 referencie esa definición.
- **Checkpoint vs. Logging:** se separó la funcionalidad de persistencia de estado (RF-016) de la de registro de ejecución (RF-015), dado que son responsabilidades de módulos distintos (`etl/checkpoint.py` y `etl/logger.py`) y tienen criterios de medición independientes.

4.2.2. Esquema de Priorización

Los requerimientos fueron clasificados en tres niveles de prioridad usando el método MoSCoW, consensuado con el director del proyecto:

Prioridad	Etiqueta MoSCoW	Criterio de clasificación
Alta	Must Have	El sistema no puede entregarse sin este requerimiento. Su ausencia impide la ejecución del pipeline o invalida los resultados de la simulación.
Media	Should Have	Requerimiento importante que mejora la usabilidad, mantenibilidad o confiabilidad del sistema, pero cuya ausencia no impide la ejecución básica del pipeline.
Baja	Could Have	Requerimiento deseable que puede diferirse a una versión futura si el tiempo de desarrollo no lo permite.

Tabla 72: Esquema de priorización MoSCoW.

En la versión 1.0 del sistema todos los requerimientos documentados tienen prioridad Alta o Media. No se identificaron requerimientos de prioridad Baja que justificaran su inclusión en el SRS; los candidatos de baja prioridad fueron documentados como requerimientos diferidos en el SPMP.

4.3. Especificación y Documentación

Cada requerimiento fue documentado siguiendo el formato Volere adaptado que se describe en la Sección 3 de este documento (ver Tabla 7: Documentación de Requerimientos). Este formato garantiza que cada requerimiento sea atómico, no ambiguo, completo y verificable, cumpliendo con las características definidas en IEEE 830-1998.

La convención de identificadores adoptada es la siguiente:

Prefijo	Tipo	Ejemplo
`RF-NNN`	Requerimiento Funcional	`RF-001`, `RF-016`
`RD-NNN`	Requerimiento de Desempeño	`RD-001`, `RD-002`
`RNF-CON-NNN`	No Funcional – Confiabilidad	`RNF-CON-001`
`RNF-DISP-NNN`	No Funcional – Disponibilidad	`RNF-DISP-001`
`RNF-SEG-NNN`	No Funcional – Seguridad	`RNF-SEG-001`
`RNF-MAN-NNN`	No Funcional – Mantenibilidad	`RNF-MAN-001`
`RNF-PORT-NNN`	No Funcional – Portabilidad	`RNF-PORT-001`
`RNF-REP-NNN`	No Funcional – Reproducibilidad	`RNF-REP-001`
`RNF-OBS-NNN`	No Funcional – Observabilidad	`RNF-OBS-001`
`RNF-USA-NNN`	No Funcional – Usabilidad	`RNF-USA-001`

Tabla 73: Convención de identificadores de requerimientos.

Los identificadores son únicos, permanentes y no se reasignan cuando un requerimiento es eliminado, para preservar la trazabilidad histórica.

4.4. Trazabilidad de Requerimientos

La trazabilidad permite rastrear cada requerimiento hacia adelante (hacia los casos de prueba y el código) y hacia atrás (hacia su fuente de origen). El equipo implementa dos niveles de trazabilidad:

4.4.1. Trazabilidad hacia el origen

Cada requerimiento está asociado a la fuente que lo originó:

Requerimiento	Fuente de origen	Artefacto de evidencia
RF-001	Estructura de columnas de la Encuesta de Movilidad 2023	Notebook 01_exploracion_encuesta.ipynb
RF-003	Documento «Base de datos con factores de ponderación y sin dobles conteos»	Acta de reunión Semana 1
RF-007	Análisis de distribución horaria de viajes (exploración Fase 1)	Notebook 02_distribucion_horaria.ipynb
RF-008	Documentación oficial SUMO 1.19 – TAZ-based OD matrix format	docs.sumo.dlr.de/docs/Demand/Importing_O/D_Matrices
RF-014	Criterio de validación acordado con el director (GEH < 5 en ≥ 85%)	Acta de reunión Semana 3
RD-001	Supuesto de hardware del SPMP (8 GB RAM, 4 núcleos)	SPMP Sección 6.3 – Supuestos y Restricciones
RNF-REP-001	Requisito científico del proyecto de grado (resultados verificables)	Acta de reunión Semana 1 – objetivo de reproducibilidad
RF-021 a RF-025	Objetivo de evaluar políticas de movilidad (VFP §2, mediano plazo: día sin carro, cierres)	demanda.py, estilos.py, cierres.py, precalculadas.py
RF-028 a RF-032	Sprint 4 de la VFP: escenarios de control de tráfico (semáforos nuevos y ajuste de fases)	intersecciones.py, programas_tls.py, priorizacion.py; docs/propuesta-semaforizacion-3d.md
RF-033 a RF-042	Solicitud del equipo: ver de forma tangible, en 3D y lado a lado, el efecto de un semáforo propuesto siguiendo a un actor	docs/propuesta-semaforizacion-3d.md; api/comparacion3d.py
RF-043	Limitación documentada del transporte público como caminata	simulation-service/README.md

Tabla 74: Trazabilidad de requerimientos seleccionados hacia su origen.

4.4.2. Trazabilidad hacia casos de prueba

La matriz de trazabilidad completa entre requerimientos y casos de prueba se documenta en la Sección 5 (Proceso de Verificación) de este documento y se mantiene actualizada en la hoja traceability_matrix.xlsx del repositorio del proyecto.

La gestión de la trazabilidad se apoya en GitHub Issues: cada requerimiento tiene un issue asociado con su identificador como etiqueta, permitiendo rastrear los commits, pull requests y casos de prueba que lo implementan y verifican.

4.5. Control de Cambios a los Requerimientos

Este proceso es complementario a la sección de Control de Requerimientos del SPMP y describe el flujo operativo que el equipo sigue cuando un requerimiento necesita ser creado, modificado o eliminado.

4.5.1. Flujo de Control de Cambios

Paso	Responsable	Acción
1. Solicitud	Cualquier integrante	Se abre un GitHub Issue con la etiqueta `change-request`, describiendo el requerimiento afectado, la razón del cambio y el impacto estimado en otros requerimientos.
2. Análisis	Responsable de Documentación + equipo	El equipo analiza el impacto del cambio sobre los requerimientos existentes, los casos de prueba y el código. El resultado se documenta en el mismo issue.
3. Aprobación	Director del proyecto	Cambios que afecten el alcance, los criterios de aceptación o los requerimientos de prioridad Alta requieren aprobación explícita del director (comentario en el issue o correo electrónico adjunto). Cambios menores (corrección de redacción, ajuste de criterios de medición) pueden ser aprobados por el equipo.
4. Implementación	Responsable de Documentación	Se actualiza la tabla del requerimiento en el SRS, se incrementa el número de versión del requerimiento y se actualiza la fecha. El historial de cambios del documento se registra en la tabla de la página inicial del SRS.
5. Cierre	Responsable de Documentación	Se cierra el issue con referencia al commit que incorpora el cambio en el SRS. Se actualiza la matriz de trazabilidad si el cambio afecta casos de prueba.

Tabla 75: Flujo de control de cambios a los requerimientos.

4.5.2. Versionamiento de Requerimientos

Cada requerimiento lleva un número de versión independiente (campo **Versión** del formato Volere). La versión inicial de todos los requerimientos de este documento es 1.0. Un cambio en la descripción o criterio de medición incrementa el número menor (1.0 → 1.1). Un cambio en el alcance o la prioridad del requerimiento incrementa el número mayor (1.0 → 2.0). Los requerimientos eliminados no se borran del documento; se marcan como **[Eliminado v X.Y]** y se conservan para preservar la trazabilidad histórica.

4.5.3. Restricciones al Control de Cambios

De acuerdo con el SPMP, se establecen las siguientes restricciones para evitar la expansión descontrolada del alcance (**scope creep**):

- No se aceptarán nuevos requerimientos funcionales de prioridad Alta después de la Semana 5 del proyecto (inicio de la Fase 3 – construcción). Los requerimientos F8 y F9 (v2.0) se incorporaron como parte del Sprint 4 (escenarios de control de tráfico) y del Sprint 5 (módulo web) definidos en la VFP; su incorporación queda sujeta a la aprobación del director según el flujo de control de cambios.
- Cambios en los criterios de aceptación de requerimientos de validación (RF-014, RNF-REP-001) requieren aprobación del director en todos los casos.
- Cualquier cambio que implique modificar los formatos de salida del pipeline (extensiones de archivo, estructura XML) debe ser evaluado contra la compatibilidad con SUMO 1.19 antes de ser aprobado.

5. Proceso de Verificación

Esta sección presenta el plan de verificación y validación que se aplicará tanto a este documento SRS como a los requerimientos individuales especificados en la Sección 3. El objetivo es garantizar que el sistema construido cumpla con lo especificado y que la especificación misma sea correcta, completa y consistente antes de iniciar la implementación.

Se distinguen dos actividades complementarias:

- **Verificación:** determinar si el sistema se está construyendo correctamente, es decir, si los artefactos producidos (código, archivos de simulación, datos intermedios) se corresponden con lo especificado en el SRS.
- **Validación:** determinar si se está construyendo el sistema correcto, es decir, si el sistema producido satisface las necesidades reales de los posibles usuarios identificados (p. ej., analisistas de movilidad, ingenieros de infraestructura, Secretaría Distrital de Movilidad Bogotá)

5.1. Verificación del Documento SRS

Antes de iniciar la fase de implementación, el documento SRS en su conjunto será sometido a un proceso de revisión estructurada para verificar que la especificación cumple con las características definidas en la Sección 3 (atomicidad, completitud, consistencia, verificabilidad, trazabilidad, entre otras).

5.1.1. Técnica de Revisión por Pares

Se realizará una sesión de revisión por pares (peer review) en la que cada integrante del equipo revisará las secciones redactadas por otro, verificando los siguientes criterios:

Criterio	Pregunta de verificación	Método
Completitud	¿Cada requerimiento tiene descripción, razón, criterio de medición, prioridad, módulo y versión?	Inspección
No ambigüedad	¿Cada requerimiento admite una única interpretación? ¿Se usan términos del glosario de la Sección 1?	Inspección
Verificabilidad	¿El criterio de medición permite comprobar el requerimiento mediante prueba, demostración, análisis o inspección?	Análisis
Consistencia	¿Algún requerimiento contradice a otro o a las restricciones de diseño definidas?	Inspección
Trazabilidad	¿Cada requerimiento tiene un identificador único y puede rastrearse hacia su fuente de origen y hacia al menos un caso de prueba?	Inspección
No redundancia	¿Existe algún requerimiento que sea duplicado o subconjunto de otro?	Inspección
Atomicidad	¿Cada requerimiento expresa una única capacidad del sistema?	Inspección

Tabla 76: Criterios de verificación del documento SRS.

Los hallazgos de la revisión se registrarán en un issue de GitHub con la etiqueta `srs-review`, indicando el identificador del requerimiento afectado, el criterio incumplido y la corrección propuesta. El documento no se considerará aprobado hasta que todos los issues de revisión estén cerrados.

5.1.2. Revisión con el Director del Proyecto

Se realizará una sesión de validación del SRS con el Ing. Andrés Calderón Romero al finalizar la revisión por pares. En esta sesión se verificará que:

- Los requerimientos de la Sección 3 reflejan correctamente las necesidades discutidas en las entrevistas de elicitación.
- Los criterios de aceptación (especialmente el umbral GEH < 5 en $\geq 85\%$ de los puntos de aforo) son correctos y consensuados.
- Las exclusiones de alcance documentadas (otras localidades, programas reales de la SDM, transporte público con GTFS) son aceptadas.
- La distribución de requerimientos por módulo es coherente con la arquitectura esperada del sistema.

5.2. Verificación de Requerimientos Individuales

Cada requerimiento de la Sección 3 será verificado mediante uno o más métodos formales, seleccionados según la naturaleza del requerimiento. Se utilizarán cuatro métodos:

Método	Descripción y aplicación en el proyecto
Prueba (T)	Ejecución del sistema o de un componente con entradas definidas y comparación de la salida con el resultado esperado. Aplicable a todos los requerimientos funcionales (RF) y de desempeño (RD). Se implementará mediante pytest con fixtures que preparan los datasets de prueba.
Demostración (D)	Ejecución del sistema completo ante el director del proyecto para mostrar que una funcionalidad opera según lo especificado. Aplicable a los requerimientos funcionales de alto nivel (F1–F5) y al requerimiento de variación paramétrica (RF-013).
Análisis (A)	Revisión del código fuente, configuración del CI o métricas generadas para verificar que el requerimiento se cumple sin necesidad de ejecutar el sistema. Aplicable a requerimientos de mantenibilidad (RNF-MAN), portabilidad (RNF-PORT) y reproducibilidad (RNF-REP).
Inspección (I)	Revisión manual de artefactos generados (archivos XML, CSV, logs) para verificar que su estructura y contenido cumplen con la especificación. Aplicable a los requerimientos de formato de salida (RF-008, RF-009, RF-010, RF-011).

Tabla 77: Métodos de verificación de requerimientos.

5.3. Plan de Casos de Prueba

A continuación se presenta el plan de casos de prueba que se diseñará e implementará para verificar los requerimientos de la Sección 3. Cada caso de prueba está asociado a uno o más requerimientos y especifica el método de verificación, las entradas de prueba y el criterio de éxito.

5.3.1. Pruebas de Requerimientos Funcionales

ID Caso	Req.	Método	Entrada de prueba	Criterio de éxito
CP-001	RF-001	T	Archivo XLSX con columnas obligatorias completas; segundo archivo XLSX sin la columna `destino_UPZ`.	Primer caso: pipeline continúa sin error. Segundo caso: pipeline termina con código de error $\neq 0$ y el log contiene «Columna requerida ausente: destino_UPZ».

ID Caso	Req.	Método	Entrada de prueba	Criterio de éxito
CP-002	RF-002	T	Ejecución sin archivo OSM local; segunda ejecución con el archivo ya presente.	Primera ejecución: archivo `usaquen.osm.pbf` generado en el directorio de datos. Segunda ejecución: log indica «Red OSM encontrada en caché, omitiendo descarga».
CP-003	RF-003	T	Dataset de 10 registros con 2 dobles conteos conocidos e identificados previamente.	Tabla de salida con exactamente 8 registros; log indica 2 registros eliminados por doble conteo con su ID_hogar.
CP-004	RF-004	T	Archivo con 1 000 registros de los cuales 50 tienen `origen_UPZ` nulo.	Log reporta 5.0% de tasa de descarte; dataset limpio contiene exactamente 950 registros.
CP-005	RF-005	T	Dataset de prueba con factores de expansión conocidos cuya suma es exactamente 12 500.0.	Suma de viajes expandidos en el output: entre 12 498.75 y 12 501.25 (±0.01%).
CP-006	RF-006	T	Dataset con viajes de todos los modos (auto, bus, moto, bicicleta).	Sistema genera un archivo `.od` por cada modo con al menos un viaje. Número de archivos `.od` = número de modos con viajes en el dataset.
CP-007	RF-007	T, I	Dataset con viajes distribuidos en los tres períodos horarios (AM, PM, nocturno).	Sistema genera al menos 3 × N archivos `.od` (N = número de modos). Los archivos AM contienen solo viajes con hora de inicio entre 6:00 y 8:59.
CP-008	RF-008	T, I	Archivo `.od` generado por el pipeline para el período AM, modo automóvil.	duarouter procesa el archivo sin errores de formato sobre la red `.net.xml` de Usaquén y genera un `.rou.xml` válido.
CP-009	RF-009	T, I	Red OSM de Usaquén recortada disponible en disco.	Archivo `.net.xml` generado por netconvert sin errores. SUMO carga

ID Caso	Req.	Método	Entrada de prueba	Criterio de éxito
				la red sin mensajes de tipo ERROR en su log.
CP-010	RF-010	T, I	Shapefile de UPZ de Usaquén y red `.net.xml` generada.	Archivo `.taz.xml` con una entrada por cada UPZ de Usaquén. Cada zona contiene al menos un nodo de la red. SUMO valida el archivo sin errores de zona vacía.
CP-011	RF-011	T, I	Ejecución de la fase F4 del pipeline con la red y matrices disponibles.	Archivo `.vtype.xml` con al menos 4 entradas (auto, bus, moto, bicicleta). SUMO carga el archivo sin advertencias de tipo desconocido.
CP-012	RF-012	T	Archivos `.od`, `.net.xml` y `.taz.xml` generados por las fases anteriores.	duarouter completa sin errores y genera `.rou.xml` con al menos un viaje por cada par O/D con valor > 0 en la matriz.
CP-013	RF-013	T, D	Dos archivos `config.yaml` idénticos excepto en el campo `factor_escala_demanda`: 1.0 y 1.2 respectivamente.	Los dos conjuntos de artefactos generados difieren en el total de viajes exactamente en un 20%. Sin modificaciones al código fuente entre ejecuciones.
CP-014	RF-014	T, I	Resultados de simulación SUMO y base de datos de aforos HMD de Usaquén.	Archivo `validacion_geh.csv` con una fila por punto de aforo disponible, columnas `punto_aforo`, `volumen_simulado`, `volumen_real`, `GEH`. Log indica el porcentaje de puntos con GEH < 5.
CP-015	RF-015	T, I	Ejecución completa del pipeline con los datos de Usaquén.	Archivo `pipeline.log` con al menos una entrada de inicio y una de fin para cada fase F1–F5, cada una con marca de tiempo.

ID Caso	Req.	Método	Entrada de prueba	Criterio de éxito
CP-016	RF-016	T	Ejecución interrumpida forzosamente al inicio de F4 (kill del proceso); relanzamiento con flag `--resume`.	Segunda ejecución salta F1–F3, inicia desde F4 y produce artefactos idénticos a una ejecución completa (verificado por hash SHA-256 de los archivos de salida).
CP-017b	RF-017	D	Simulación con resultados disponibles; abrir el módulo web en el navegador y seleccionar la simulación.	Las métricas de la simulación se renderizan en menos de 3 segundos. Todos los campos (volumen, tiempo, GEH) muestran valores distintos de cero.
CP-018b	RF-018	T	Petición GET a /simulaciones/{id}/metricas con un ID válido y con un ID inexistente.	ID válido: respuesta HTTP 200 con JSON. ID inexistente: respuesta HTTP 404.
CP-019b	RF-019	D	Abrir el formulario de parametrización, seleccionar período PM y factor 1.2, confirmar.	El pipeline se ejecuta con los parámetros seleccionados. El módulo web muestra indicador de progreso y mensaje de éxito al finalizar.
CP-020b	RF-020	D	Ingresar factor de escala igual a 0 en el formulario y pulsar Confirmar.	El formulario muestra mensaje de error visible. No se envía ninguna petición al backend.
CP-029	RF-021, RF-022	T	`test_demanda.py`: preset `encuesta` con conteos por defecto; presets alternos; conteos por modo.	La demanda de `encuesta` reproduce la expansión simple; los presets conservan el total; los conteos se cumplen por modo.
CP-030	RF-025	T	`test_precalculadas.py`: generar una precalculada dos veces con los mismos insumos.	Mismo `trips.geojson.gz` (gzip con `mtime=0`) y mismo hash de insumos en `meta.json`.
CP-031	RF-026	T	`test_api.py` con pipeline falso inyectado en `GestorCorridas`:	El avance es monótono; tras cancelar solo queda

ID Caso	Req.	Método	Entrada de prueba	Criterio de éxito
			lanzar, consultar y cancelar.	`estado.json` con estado `cancelada`.
CP-032	RF-028	T	`test_intersecciones.py`: red de prueba con `tlLogic` y conexiones.	Un semáforo por `tlLogic` con sus accesos, nombres de vía y volúmenes por acceso.
CP-033	RF-029	T, D	Recortar 300 m alrededor de un cruce con una corrida que tenga vehroute.	`zona.net.xml` válido para SUMO y `zona.rou.xml` con los viajes que cruzan la zona.
CP-034	RF-030	T, D	Guardar un programa con una fase de largo incorrecto; calcular Webster con volúmenes de la ficha.	El primero se rechaza indicando la fase; el ciclo de Webster es ≥ 30 s.
CP-035	RF-033	T	`test_semaforo_nuevo.py`: nodo sin semáforo, nodo con semáforo y nodo inexistente; `test_demanda_zona.py`: argumentos de `randomTrips.py` por modo.	Se instala el semáforo solo en el nodo válido; los otros fallan con mensaje claro; el período de cada modo es 3600 / intensidad.
CP-036	RF-033, RF-034	T	`test_comparacion3d.py` (gestor con preparación falsa) y `test_escena3d.py` (edificios y geometría).	Estados `preparando` → `lista`/`error`; límite de 2 comparaciones (429); geometría con edificios recortados al bbox.
CP-037	RF-035, RF-036, RF-040	D	Abrir una comparación sobre un cruce prioritario, lanzar una moto agresiva y luego un peatón por accesos distintos.	El actor aparece en ambos lados, la cámara lo sigue y el HUD muestra velocidad, acción, decisiones y métricas distintas por lado. Verificado por integración vía el proxy del visor: 149 de 150 frames con el actor en ambos lados.
CP-038	RF-037	T	`test_telemetria.py`: TTC, DRAC, PET, niveles y clasificación de acciones.	TTC = 1 s → índice 100 (alto); sin conflictos → 0 (bajo); cada combinación produce la acción esperada.

ID Caso	Req.	Método	Entrada de prueba	Criterio de éxito
CP-039	RF-038, RF-039	D	Seguir un actor y elegir una vía de la lista.	El minimapa sigue al actor y deja su rastro; la lista de la vía solo contiene actores sobre sus edges.

Tabla 78: Plan de casos de prueba para requerimientos funcionales.

5.3.2. Pruebas de Requerimientos No Funcionales

ID Caso	Req.	Método	Procedimiento	Criterio de éxito
CP-017	RD-001	T	Ejecutar el pipeline completo (F1–F5) en la máquina de referencia (8 GB RAM, 4 núcleos) con el dataset completo de Usaquén. Registrar tiempo de inicio y fin.	Tiempo total de ejecución ≤ 14 400 segundos (4 horas).
CP-018	RD-002	T	Ejecutar únicamente la fase F2 sobre el dataset completo de la encuesta (500 000 registros) en la máquina de referencia.	Tiempo de ejecución de F2 ≤ 1 800 segundos (30 minutos).
CP-019	RNF-CON-001	T	Archivo de prueba con 10% de registros con NaN en `origen_UPZ`.	Pipeline completa sin excepciones no controladas. Log reporta exactamente 10% de descarte en F1.
CP-020	RNF-CON-002	T	Fallo forzado (excepción lanzada) al inicio de F4.	Directorio de salida contiene artefactos de F1–F3. Log indica «Fase F4 fallida» con la causa del error.
CP-021	RNF-DISP-001	T	Ejecutar fases F2–F5 con la interfaz de red deshabilitada (modo avión o desconexión del adaptador).	Pipeline completa sin errores de conexión en ninguna de las fases F2–F5.
CP-022	RNF-SEG-001	I	Inspeccionar el contenido de los archivos `.od`, `.rou.xml` y `.taz.xml` generados con el dataset de Usaquén.	Ningún archivo de salida contiene el campo `ID_hogar` ni ningún identificador individual de la encuesta.
CP-023	RNF-MAN-001	A	Ejecutar `pydocstyle` sobre el paquete `etl/` al finalizar la implementación.	Cero funciones públicas sin docstring completo (descripción, parámetros, retorno, ejemplo).
CP-024	RNF-MAN-002	A	Verificar los logs del workflow de GitHub Actions en la rama principal tras un commit de código.	El step de pylint reporta score ≥ 8.0/10. El build está marcado como exitoso.

ID Caso	Req.	Método	Procedimiento	Criterio de éxito
CP-025	RNF-PORT-001	A	Revisar los logs de los jobs de CI en GitHub Actions para Ubuntu 22.04 y Windows 10.	Ambos jobs completan exitosamente. Ningún step falla por incompatibilidad de sistema operativo.
CP-026	RNF-PORT-002	I	Inspeccionar el archivo `requirements.txt` del repositorio.	Todas las entradas siguen el formato `biblioteca==X.Y.Z`. No existen rangos del tipo >= ni ~=.
CP-027	RNF-REP-001	T, A	Ejecutar el pipeline dos veces consecutivas con los mismos archivos de entrada y el mismo `config.yaml`. Calcular el hash SHA-256 de cada archivo de salida en ambas ejecuciones.	El hash SHA-256 de cada archivo de salida es idéntico entre las dos ejecuciones. El script `scripts/verify_reproducibility.py` reporta 0 diferencias.
CP-028	RNF-OBS-001	T	Ejecutar la fase F2 con el dataset completo y medir el tiempo entre mensajes de progreso en consola.	Al menos un mensaje de progreso visible en consola por cada minuto de ejecución de F2.
CP-040	RD-003	T	Script de integración: preparar una comparación sobre un cruce real y consumir 150 frames por WebSocket.	Preparación ≤ 60 s y ≥ 4 frames/s a 1×. Resultado medido: 14–40 s y 5 frames/s.
CP-041	RD-004	T	Perfil de rendimiento del navegador durante 60 s de comparación.	≥ 30 cuadros por segundo en la máquina de referencia.
CP-042	RNF-USA-001	D	Sesión con el director: lanzar una corrida, abrir una intersección y una comparación 3D sin asistencia.	Las tres tareas se completan sin ayuda del equipo.

Tabla 79: Plan de casos de prueba para requerimientos no funcionales.

5.4. Criterios de Aceptación del Sistema

El sistema se considerará verificado y listo para la entrega al director del proyecto cuando se cumplan simultáneamente los siguientes criterios:

1. Todos los casos de prueba de requerimientos funcionales (CP-001 a CP-016 y CP-029 a CP-039) aplicables a la versión entregada han sido ejecutados y su resultado es **Pasa**. La suite automatizada de `simulation-service/test/` (99 pruebas en v2.0) pasa completa.
2. Los casos de prueba de desempeño (CP-017 y CP-018) han sido ejecutados en la máquina de referencia definida en Tabla 4 y su resultado es **Pasa**.
3. Al menos el 85% de los puntos de aforo HMD de Usaquén obtienen un GEH < 5 en la simulación de validación (criterio del requerimiento RF-014).

4. Los jobs de CI en GitHub Actions para Ubuntu y Windows están en estado **verde** (todos los steps exitosos) en el último commit de la rama principal.
5. El script `scripts/verify_reproducibility.py` reporta 0 diferencias entre dos ejecuciones consecutivas del pipeline con los mismos datos de entrada.
6. No existen issues abiertos en GitHub con la etiqueta `blocker` o `bug-critico`.

5.5. Proceso de Registro de Defectos

Durante la fase de verificación, los defectos encontrados se registrarán y gestionarán siguiendo el proceso definido a continuación:

Paso	Responsable	Acción
1. Reporte	Responsable de Pruebas	Se abre un GitHub Issue con la etiqueta `bug`, indicando el caso de prueba fallido, el requerimiento afectado, los pasos para reproducir el defecto, el resultado obtenido y el resultado esperado.
2. Clasificación	Responsable de Pruebas	Se asigna una severidad: `blocker` (impide la entrega), `bug-critico` (afecta un requerimiento de prioridad Alta), `bug-menor` (afecta un requerimiento de prioridad Media). Los blockers y críticos deben corregirse antes de la entrega.
3. Asignación	Responsable de Documentación	El issue se asigna al integrante del equipo responsable del módulo afectado según la distribución de requerimientos de la Sección 2.8.
4. Corrección	Desarrollador asignado	Se implementa la corrección en una rama separada y se abre un pull request referenciando el issue. El PR debe incluir o actualizar el caso de prueba que detectó el defecto.
5. Verificación	Responsable de Pruebas	Se ejecuta nuevamente el caso de prueba fallido y los casos relacionados para confirmar la corrección y verificar la ausencia de regresiones.
6. Cierre	Responsable de Pruebas	Se cierra el issue con referencia al commit de corrección y se actualiza el estado del caso de prueba en la matriz de trazabilidad.

Tabla 80: Proceso de registro y gestión de defectos.

6. Anexos

6.1. Descripción de la Encuesta de Movilidad de Bogotá 2023

La **Encuesta de Movilidad de Bogotá 2023** es el instrumento oficial de caracterización de los patrones de desplazamiento de la población capitalina. Es llevada a cabo por la Secretaría Distrital de Movilidad y constituye la principal fuente de datos de demanda del presente proyecto.

Atributo	Descripción
Cobertura geográfica	Las 20 localidades urbanas del Distrito Capital de Bogotá.
Unidad de muestreo	Hogar residente en Bogotá.

Variables de viaje	Modo de transporte, hora de salida y llegada, propósito del viaje, zona origen y zona destino (codificadas por UPZ).
Variables socioeconómicas	Tamaño del hogar, estrato socioeconómico, tenencia de vehículo particular y bicicleta.
Factor de expansión	Ponderador estadístico que permite proyectar la muestra al universo poblacional de la ciudad.
Uso en el proyecto	Construcción de matrices O/D por UPZ para la hora de máxima demanda AM, segmentadas por modo de transporte.

6.2. Posición de Bogotá en el TomTom Traffic Index 2025

El **TomTom Traffic Index** es un informe anual que clasifica ciudades del mundo según su nivel de congestión vehicular, medido como el tiempo adicional de viaje respecto a condiciones de tráfico libre.

Indicador	**Valor reportado (2025)**
Posición global	7.° lugar entre las ciudades más congestionadas del mundo.
Índice de congestión	69,9 %.
Velocidad promedio en hora pico	Aproximadamente 15 km/h.
Implicación para el proyecto	El alto nivel de congestión evidencia la necesidad de herramientas de simulación que permitan evaluar políticas de movilidad antes de su implementación.